LAPORAN TUGAS AKHIR I

PERBANDINGAN PENALARAN DAN KOMUNIKASI BERBASIS RUANG LATEN PADA SISTEM MULTI-AGEN: *MIXTURE OF INPUTS* TERHADAP METODE PENALARAN LATEN LAIN DAN KOMUNIKASI BERBASIS TEKS

Harun Yahya
22/505239/PA/21749

PEMBIMBING : Atina Husnaqilati, S.Si., M.Sc., Ph.D.

PROGRAM SARJANA PROGRAM STUDI STATISTIKA
DEPARTEMEN MATEMATIKA
FAKULTAS MATEMATIKA DAN ILMU PENGETAHUAN ALAM
UNIVERSITAS GADJAH MADA
YOGYAKARTA

2026

INTISARI

Komunikasi antaragen dalam *Multi-Agent System* (MAS) berbasis *Large Language Model* (LLM) umumnya menggunakan teks, sedangkan komunikasi berbasis ruang laten berpotensi mengurangi kebutuhan representasi tekstual. Namun, bagaimana formulasi penalaran laten dan medium komunikasi memengaruhi kinerja MAS masih perlu dikaji. Penalaran laten memungkinkan LLM memproses informasi melalui representasi internal, sementara komunikasi laten melalui *key-value cache* (KV-cache) membuka peluang bagi agen untuk bertukar informasi tanpa harus mengubahnya menjadi teks terlebih dahulu. *Mixture of Inputs* (MoI) menggunakan pendekatan Bayesian untuk menggabungkan beberapa kemungkinan representasi berdasarkan bobot posterior, sehingga diharapkan dapat menghasilkan masukan yang lebih adaptif terhadap informasi yang tersedia dan mendukung proses penalaran berikutnya. Penelitian ini membandingkan empat formulasi penalaran laten, yaitu *raw*, *soft*, *sample*, dan MoI, serta dua medium komunikasi, yaitu teks dan KV-cache, menggunakan model *open-weight* Qwen3-8B. Pengujian dilakukan pada *benchmark* GSM8K, ARC-Challenge, dan HumanEval+ dengan mengevaluasi akurasi penyelesaian tugas, penggunaan token, dan waktu inferensi. Perbandingan ini diharapkan dapat menunjukkan sejauh mana komunikasi berbasis KV-cache meningkatkan efisiensi dibandingkan komunikasi berbasis teks, sekaligus mengungkap pengaruh formulasi penalaran laten terhadap keseimbangan antara efisiensi dan akurasi.

Kata kunci: *Mixture of Inputs*, penalaran laten, *Multi-Agent System*, KV-cache, *Large Language Model*.

DAFTAR ISI

DAFTAR ISI . . . . . **i**
BAB I PENDAHULUAN **2**
1.1. Latar Belakang 2
1.2. Pembatasan Masalah 5
1.3. Tujuan dan Manfaat Penelitian 5
1.4. Tinjauan Pustaka 6
1.5. Metodologi Penelitian 7
1.6. Sistematika Penulisan 8
BAB II DASAR TEORI **10**
2.1. Variabel Random 10
2.1.1. Variabel Random Diskret dan Kontinu 10
2.2. Ekspektasi dan Variansi 12
2.2.1. Ekspektasi Variabel Random 12
2.2.2. Ekspektasi Vektor Random 13
2.2.3. Variansi dan Kovariansi 13
2.2.4. Koefisien Korelasi 14
2.3. Matriks 14
2.3.1. Jenis dan Operasi Matriks 14
2.4. Vektor 16
2.4.1. Hasil Kali Dalam dan Norma Euklides 16
2.4.2. Kosinus Sudut Antar-Vektor 17
2.5. Tensor 17
2.6. Distribusi 18
2.6.1. Distribusi Multinomial 18
2.6.2. Distribusi Kategorik dan Vektor Indikator 19
2.6.3. Distribusi Beta 20
2.6.4. Distribusi Dirichlet 21
2.7. Himpunan Konveks 22
2.7.1. Himpunan Konveks 22
2.7.2. Kombinasi Konveks dan Convex Hull 23
2.7.3. Simpleks Probabilitas 24
2.8. Entropi 25
2.8.1. Entropi Shannon 25
2.8.2. Batas Entropi 25
2.9. Inferensi Bayes 26
2.9.1. Definisi Bayes: Prior, Likelihood, dan Posterior 26
2.9.2. Prior Konjugat 27

2.9.3. Kekonjugatan Dirichlet–Multinomial. 27
2.9.4. Rerata Posterior sebagai Estimator Bayes 30
2.10. Model Bahasa dan Arsitektur *Transformer*. 32
2.10.1. Model Bahasa Autoregresif. 32
2.10.2. Jaringan Saraf, Fungsi Aktivasi, dan Fungsi *Softmax*. 33
2.10.3. Mekanisme Atensi dan *KV-Cache* 34
2.11. Strategi Pengambilan Sampel Token 37
2.12. Sistem Multi-Agen 39
2.12.1. Definisi Agen dan Sistem Multi-Agen 39
2.12.2. Topologi Sekuensial dan Hierarkis. 40
BAB III METODOLOGI PENELITIAN 41
3.1. Arsitektur Qwen3-8B 41
3.1.1. *Hidden State* dan Pembentukan Representasi Laten. 42
3.1.2. *Grouped-Query Attention* dan Dimensi *Key-Value* 44
3.1.3. *Key-Value Cache* (KV-cache) 46
3.2. Komunikasi Antar-Agen. 47
3.2.1. Komunikasi `text` 48
3.2.2. Komunikasi `kv`. 49
3.3. Langkah Laten 53
3.3.1. Satu Latent Pass 54
3.3.2. Penghentian Langkah Laten 55
3.3.3. Kerangka Metode 56
3.4. Metode Pembentukan Langkah Laten 57
3.4.1. Metode `raw` 58
3.4.2. Metode `soft` 60
3.4.3. Metode `sample` 61
3.4.4. Metode `moi` 62
3.5. Rancangan Eksperimen dan Analisis. 69
3.5.1. Rancangan Eksperimen 69
3.5.2. Pengukuran dan Analisis Kinerja 71
3.5.3. Benchmark dan Metrik Evaluasi. 74
3.5.4. Pengukuran Geometris. 75
3.5.5. Interpolasi Geometris. 80
DAFTAR PUSTAKA 87

BAB I

PENDAHULUAN

1.1. Latar Belakang

Beberapa tahun terakhir ditandai oleh perkembangan kecerdasan buatan (*artificial intelligence*, AI) yang berlangsung sangat pesat, baik pada layanan komersial berbayar maupun pada model *open-weight*. Kapasitas komputasi AI global tumbuh rata-rata 3,3 kali per tahun sejak 2022 dan mencapai 17,1 juta *H100-equivalents* [Stanford HAI, 2026], sementara biaya inferensi untuk kemampuan yang setara dengan GPT-3.5 turun lebih dari 280 kali antara November 2022 dan Oktober 2024 [Stanford HAI, 2025].

Large Language Model (LLM) menghasilkan teks secara autoregresif. Pada setiap langkah, model menghitung distribusi peluang atas seluruh kosakata, memilih satu *token* diskret dari distribusi tersebut, menambahkannya ke rangkaian masukan, kemudian mengulangi proses yang sama untuk menghasilkan *token* berikutnya [Vaswani dkk, 2017]. Kemampuan penalaran model juga dibangun di atas mekanisme tersebut. Teknik *chain-of-thought* mendorong model untuk menghasilkan langkah-langkah penalaran dalam bentuk teks sebelum menghasilkan jawaban akhir [Wei dkk, 2022], sehingga setiap langkah penalaran tetap diwujudkan melalui pemilihan *token* diskret.

Peran LLM kemudian berkembang dari sistem percakapan menjadi *agent*. Model kini dapat melakukan pencarian informasi, terhubung dengan sumber data dan lingkungan eksternal melalui *Model Context Protocol* (MCP), memanggil *tools*, hingga mengoperasikan antarmuka komputer. *Agent* merupakan sistem yang menerima suatu tujuan, menyusun rencana, dan melakukan tindakan sesuai lingkungan secara relatif otonom [Russell dan Norvig, 2021].

Beberapa agen kemudian dapat digabungkan ke dalam satu *Multi-Agent System* (MAS). Setiap agen dapat diberikan peran yang berbeda dan saling bertukar

informasi untuk menyelesaikan suatu tugas, sehingga keseluruhan sistem dapat dipandang sebagai suatu bentuk kolaborasi antarsistem yang menyerupai tim kerja [Wooldridge dan Jennings, 1995]. MAS berbasis LLM telah digunakan dalam berbagai tugas, termasuk penalaran matematika dan sains, *question answering*, penggunaan *tools*, hingga pengambilan keputusan [Guo dkk, 2024].

Peningkatan kemampuan tersebut juga diikuti oleh peningkatan kebutuhan sumber daya komputasi. Model dengan kemampuan yang lebih tinggi pada umumnya membutuhkan kapasitas memori dan GPU yang lebih besar, sedangkan pada MAS suatu persoalan dapat diselesaikan melalui beberapa kali proses inferensi. Pada layanan komersial berbasis API, penggunaan model umumnya dikenai biaya berdasarkan jumlah *token* masukan dan keluaran [OpenAI, 2026, Anthropic, 2025]. Sementara itu, model *open-weight* yang dijalankan pada GPU lokal maupun sewaan menimbulkan biaya berdasarkan durasi penggunaan sumber daya komputasi. Dengan demikian, pada kedua skema tersebut, pengurangan jumlah *token* yang diproses dan waktu inferensi berpotensi menurunkan biaya penggunaan.

Pertimbangan lain dalam penggunaan LLM adalah privasi dan keamanan data, karena penggunaan layanan tertutup mengharuskan data dikirim dan diproses melalui infrastruktur pihak ketiga. Di antara organisasi yang mengalami insiden keamanan terkait AI, 97% menyatakan tidak memiliki kontrol akses AI yang memadai, sementara 63% belum memiliki kebijakan tata kelola AI [IBM, 2025]. Model *open-weight* yang dijalankan pada infrastruktur yang dikendalikan sendiri menjadi salah satu alternatif yang menarik, meskipun kemampuan model tersebut pada kondisi tertentu masih dapat berada di bawah model komersial.

Pada MAS berbasis LLM, komunikasi antaragen umumnya berlangsung melalui teks, yaitu keluaran satu agen dituliskan sebagai rangkaian *token* diskret sebelum digunakan sebagai masukan bagi agen berikutnya [Zou dkk, 2025]. Pendekatan tersebut memiliki keunggulan dari sisi keterbacaan dan kemudahan interpretasi, tetapi memerlukan proses *decoding* untuk menghasilkan teks dan pemrosesan kembali ketika teks tersebut digunakan sebagai masukan bagi agen berikutnya. Akibatnya, jumlah *token* yang diproses dan waktu inferensi dapat meningkat seiring

dengan bertambahnya panjang penalaran dan jumlah agen yang terlibat.

Sejumlah penelitian kemudian berupaya mengurangi ketergantungan terhadap representasi token diskret pada dua bagian yang berbeda. Pada tingkat penalaran dalam satu agen, *Soft Thinking* [Zhang dkk, 2025] dan LatentMAS [Zou dkk, 2025] mengganti pemilihan token diskret dengan representasi pada ruang kontinu atau ruang laten. Pada *Soft Thinking*, masukan untuk langkah berikutnya dibentuk dari distribusi peluang token, sedangkan pada LatentMAS representasi tersebut dapat dibentuk langsung dari *hidden state*. Pada tingkat komunikasi antaragen, LatentMAS meneruskan KV-*cache* sebagai representasi keadaan internal dari satu agen ke agen berikutnya. Pendekatan-pendekatan tersebut berpotensi mengurangi penggunaan *token* dan waktu inferensi, tetapi menghilangkan atau mengurangi representasi token diskret juga dapat memengaruhi akurasi sistem.

Mixture of Inputs (MoI) [Zhuang dkk, 2025] menempuh pendekatan yang berbeda dengan mempertahankan kedua sumber informasi tersebut, yaitu distribusi peluang keluaran model dan token yang terpilih melalui proses *sampling*. Dengan demikian, informasi mengenai kemungkinan token lain yang tersedia dalam distribusi model tidak sepenuhnya hilang setelah satu token dipilih. Penggabungan kedua sumber informasi tersebut diturunkan melalui pendekatan inferensi Bayes.

Penelitian ini menguji kedua aspek tersebut secara bersamaan dalam satu rancangan eksperimen. Pada aspek penalaran, MoI [Zhuang dkk, 2025] ditetapkan sebagai metode utama dan dibandingkan dengan tiga cara lain dalam membentuk representasi langkah penalaran, yaitu *raw* sebagaimana digunakan oleh LatentMAS, *soft*, dan *sample* yang tetap memilih satu token. Pada aspek komunikasi, komunikasi berbasis teks dibandingkan dengan komunikasi berbasis KV-*cache* sebagaimana digunakan oleh LatentMAS.

Evaluasi dilakukan menggunakan model *open-weight* Qwen3-8B [Yang dkk, 2025] pada tiga *benchmark*, yaitu GSM8K [Cobbe dkk, 2021], ARC-Challenge [Clark dkk, 2018], dan HumanEval+ [Liu dkk, 2023]. Seluruh perlakuan dilakukan secara *training-free* dan menggunakan konfigurasi bawaan masing-masing metode tanpa pencarian parameter. Dengan demikian, penelitian ini diarahkan untuk men-

jawab apakah Multi-Agent System dapat dibuat lebih efisien dalam penggunaan *token* dan waktu inferensi dengan tetap mempertahankan tingkat akurasinya.

1.2. Pembatasan Masalah

Agar penelitian tetap terarah dan tidak meluas di luar ruang lingkup yang ditetapkan, penelitian ini dibatasi pada hal-hal berikut.

1. Model bahasa yang digunakan adalah Qwen3-8B dengan inferensi lokal. Penelitian tidak menguji generalisasi terhadap ukuran maupun keluarga model bahasa lainnya.

2. Seluruh intervensi terhadap proses penalaran bersifat *training-free*. Tidak dilakukan perubahan bobot model, *fine-tuning*, maupun optimasi lainnya.

3. Metode penalaran laten yang dibandingkan dibatasi pada *raw*, *soft*, *sample*, dan *Mixture of Inputs* (MoI).

4. Medium komunikasi utama yang dibandingkan adalah komunikasi berbasis teks dan komunikasi laten berbasis KV-*cache*.

5. Evaluasi kemampuan penalaran dibatasi pada GSM8K, ARC-Challenge, dan HumanEval-Plus. Ketiga tugas digunakan untuk merepresentasikan keluaran numerik, pilihan simbolik, dan penulisan kode.

1.3. Tujuan dan Manfaat Penelitian

Sejalan dengan latar belakang yang telah diuraikan, penelitian ini bertujuan untuk:

1. Merumuskan *Mixture of Inputs* bersama tiga cara lain membentuk langkah penalaran laten, yaitu *raw*, *soft*, dan *sample*, ke dalam satu kerangka yang sama sehingga keempatnya dapat dibandingkan secara adil pada model Qwen3-8B, baik pada medium komunikasi berbasis teks maupun KV-*cache*.

2. Menganalisis efisiensi penggunaan token dan waktu inferensi dari masing-masing metode komunikasi dan penalaran laten, baik antara komunikasi berbasis teks dengan KV-*cache* maupun antar-metode KV-*cache* itu sendiri.

3. Menganalisis hubungan antara efisiensi dan akurasi untuk mengetahui metode yang memberikan keseimbangan paling baik antara pengurangan biaya komputasi dan kemampuan penyelesaian tugas.

Manfaat yang diharapkan adalah sebagai berikut.

1. Memberikan kontribusi bagi kajian komunikasi antaragen berbasis LLM, khususnya mengenai penggunaan representasi kontinu dan ruang laten sebagai alternatif penalaran berbasis token diskret, sekaligus bahan pembanding bagi penelitian lanjutan yang mengembangkan metode komunikasi laten pada Multi-Agent System.

2. Menyediakan perbandingan beberapa metode komunikasi dan representasi informasi dalam kondisi eksperimen yang terkendali, yaitu model dasar dan *benchmark* yang sama sehingga karakteristik masing-masing metode dari segi akurasi, penggunaan token, dan waktu inferensi dapat dipahami.

3. Mengetahui apakah pengurangan komunikasi berbasis teks melalui representasi laten, khususnya *Mixture of Inputs*, dapat menghasilkan Multi-Agent System yang lebih cepat dan lebih murah tanpa mengorbankan akurasi secara signifikan, sebagai pertimbangan praktis dalam merancang sistem yang dijalankan pada model *open-weight* di infrastruktur lokal maupun *cloud* GPU.

1.4. Tinjauan Pustaka

Berikut beberapa literatur kunci yang menjadi acuan penelitian ini.

Zou dkk [2025] mengusulkan LatentMAS, kerangka kolaborasi *multi-agent* di ruang laten dengan dua mekanisme yang dipakai penelitian ini. Pertama, menghasilkan pikiran laten di dalam satu agen: model menambahkan representasi lapisan terakhir secara autoregresif sebagai pengganti langkah pemilihan token, lalu

menyelaraskannya kembali ke ruang *embedding* masukan melalui pemetaan linear berbasis regresi *ridge*. Kedua, pemindahan memori kerja antaragen: KV-*cache* seluruh lapisan diteruskan dari satu agen ke agen berikutnya tanpa penulisan teks. Penelitian ini memakai mekanisme kedua sebagai medium komunikasi laten, dan mekanisme pertama sebagai salah satu formulasi penalaran yang dibandingkan (*raw*).

Zhang dkk [2025] mengusulkan *soft thinking*, yang mengganti pemilihan token diskret selama penalaran dengan *concept token* berupa kombinasi berbobot *embedding* token menurut distribusi peluangnya. Wu dkk [2025] menelaah mekanisme kerjanya dan menunjukkan bahwa penalaran lunak yang deterministik cenderung terkunci pada token berpeluang tertinggi. Penelitian ini membandingkan kedua sisi tersebut secara langsung: formulasi *soft* yang mempertahankan seluruh distribusi dan formulasi *sample* yang mengambil satu token melalui *sampling*.

Zhuang dkk [2025] mengusulkan *Mixture of Inputs* dan memandang pembentukan masukan bagi langkah berikutnya sebagai persoalan inferensi Bayes. Distribusi peluang keluaran model diperlakukan sebagai parameter konsentrasi *prior* Dirichlet dan token hasil pengambilan sampel sebagai satu observasi multinomial, sehingga melalui kekonjugatan Dirichlet–Multinomial diperoleh *posterior* berbentuk tertutup. Masukan bagi langkah berikutnya dibentuk dari ekspektasi *posterior* $\mathbb{E}[w \mid \cdot]$ atas bobot campuran tersebut, dengan besar kontribusi *prior* relatif terhadap observasi diatur oleh entropi distribusi, yaitu oleh tingkat keyakinan model, sehingga metode tersebut menyesuaikan diri tanpa penalaan parameter. MoI dirancang untuk penalaran satu model dan belum diuji sebagai mekanisme komunikasi pada sistem *multi-agent*; celah itulah yang menjadi titik tolak penelitian ini.

Hao dkk [2024] dan survei Chen dkk [2025] menempatkan ketiga pendekatan di atas dalam paradigma penalaran laten yang lebih luas, yaitu pemindahan langkah penalaran dari barisan token diskret ke ruang representasi kontinu.

1.5. Metodologi Penelitian

Metodologi yang digunakan adalah studi literatur dan eksperimen terkendali. Studi literatur dilakukan dengan menelaah publikasi mengenai penalaran laten,

kolaborasi *multi-agent*, dan evaluasi model bahasa, kemudian menurunkan persamaan langkah laten tiap metode dari sumber aslinya ke dalam satu kerangka notasi yang sama. Eksperimen dilakukan dengan menjalankan sistem *multi-agent* berbasis model *open-weight* Qwen3-8B pada tolok ukur GSM8K, ARC-Challenge, dan HumanEval+. Medium komunikasi antaragen dan formulasi representasi laten diperlakukan sebagai faktor yang divariasikan, sedangkan model, topologi agen, *prompt*, banyaknya langkah laten, suhu. Seluruh kondisi dievaluasi pada soal yang sama sehingga perbandingannya bersifat berpasangan.

1.6. Sistematika Penulisan

Penyusunan laporan ini mengacu pada sistematika berikut.

BAB I PENDAHULUAN

Bab ini berisi latar belakang, pembatasan masalah, tujuan dan manfaat penelitian, tinjauan pustaka, metodologi penelitian, dan sistematika penulisan.

BAB II LANDASAN TEORI

Bab ini menguraikan dasar teori yang digunakan, mencakup probabilitas dan distribusi yang relevan, teori informasi, inferensi Bayes, aljabar linear dan representasi vektor, arsitektur *transformer* beserta KV-*cache*, penalaran laten dan keempat formulasi pembentukan representasi laten, sistem *multi-agent*, konsep evaluasi empiris model bahasa, dan pengujian statistik yang dipakai.

BAB III METODOLOGI PENELITIAN

Bab ini menetapkan objek penelitian berupa persamaan langkah laten, menurunkan keempat persamaan yang dibandingkan ke dalam satu kerangka bersama, menyatakan rancangan dua sumbu eksperimen, hipotesis, data dan tugas evaluasi, protokol pengambilan sampel berpasangan, metrik, serta prosedur analisis statistik.

BAB IV HASIL DAN PEMBAHASAN

Bab ini menyajikan hasil eksperimen beserta pembahasannya, mencakup perbandingan akurasi antar-persamaan, keterbedaan hasil menurut jenis tugas, biaya komputasi medium laten, fidelitas simbolik keluaran, bukti mekanisme yang

menjelaskan pola hasil, serta batas berlaku seluruh temuan.

BAB V KESIMPULAN DAN SARAN

Bab ini memuat kesimpulan yang menjawab tujuan penelitian beserta saran bagi penelitian selanjutnya.

BAB II

DASAR TEORI

2.1. Variabel Random

Variabel random memetakan setiap hasil suatu percobaan acak ke sebuah bilangan real sehingga hasil percobaan dapat diperlakukan secara kuantitatif.

Definisi 2.1.1 (Bain dan Engelhardt, 1992) *Variabel random X adalah fungsi dengan daerah asal ruang sampel S yang memetakan setiap anggota S ke bilangan real,*

$$X : S \longrightarrow \mathbb{R}, \qquad X(e) = x, \quad e \in S,\ x \in \mathbb{R}.$$

Variabel random dinotasikan dengan huruf kapital X, Y, Z, sedangkan nilai yang mungkin diambil dinotasikan dengan huruf kecil x, y, z. Berdasarkan himpunan nilai yang mungkin diambil, variabel random dibedakan menjadi diskret dan kontinu.

2.1.1. Variabel Random Diskret dan Kontinu

Variabel random dibedakan menjadi variabel random diskret dan kontinu berdasarkan himpunan nilai yang mungkin diambil serta cara distribusi probabilitasnya dinyatakan.

Definisi 2.1.2 (Bain dan Engelhardt, 1992) *Variabel random X disebut diskret apabila himpunan semua nilai yang mungkin diambil, $\{x_1, x_2, x_3, \ldots\}$, merupakan himpunan terhitung (countable).*

Untuk variabel random diskret, distribusi probabilitas dinyatakan melalui fungsi massa peluang (*probability mass function*, pmf).

Definisi 2.1.3 (Bain dan Engelhardt, 1992) *Fungsi massa peluang dari variabel*

random diskret X adalah fungsi p_X yang didefinisikan sebagai

$$p_X(x) = P(X = x), \qquad x \in \mathcal{X}, \tag{2.1}$$

dengan $\mathcal{X}$ merupakan himpunan nilai yang mungkin diambil oleh X. Fungsi tersebut memenuhi

$$p_X(x) \geq 0, \qquad \sum_{x \in \mathcal{X}} p_X(x) = 1. \tag{2.2}$$

Sementara itu, variabel random kontinu didefinisikan melalui fungsi distribusi kumulatifnya yang dapat dinyatakan sebagai integral suatu fungsi kepadatan peluang.

Definisi 2.1.4 (Bain dan Engelhardt, 1992) *Variabel random X disebut kontinu apabila terdapat fungsi f_X, yang disebut fungsi kepadatan peluang (probability density function, pdf), sedemikian sehingga fungsi distribusi kumulatifnya memenuhi*

$$F_X(x) = \int_{-\infty}^{x} f_X(t)\, dt, \qquad x \in \mathbb{R}. \tag{2.3}$$

Definisi 2.1.5 (Bain dan Engelhardt, 1992) *Fungsi f_X merupakan fungsi kepadatan peluang dari variabel random kontinu X jika memenuhi*

$$f_X(x) \geq 0, \qquad x \in \mathbb{R}, \tag{2.4}$$

dan

$$\int_{-\infty}^{\infty} f_X(x)\, dx = 1. \tag{2.5}$$

Peluang bahwa X berada pada interval $[a, b]$, dengan $a \leq b$, diperoleh melalui

$$P(a \leq X \leq b) = \int_{a}^{b} f_X(x)\, dx. \tag{2.6}$$

2.2. Ekspektasi dan Variansi

2.2.1. Ekspektasi Variabel Random

Ekspektasi merupakan salah satu ukuran yang digunakan untuk menunjukkan nilai rata-rata yang diharapkan dari suatu variabel random. Untuk variabel random diskret, ekspektasi diperoleh melalui penjumlahan nilai-nilai yang mungkin, sedangkan untuk variabel random kontinu digunakan integral.

Definisi 2.2.1 (Subanar, 2013) *Ekspektasi atau expected value dari variabel random X dapat ditulis sebagai*

$$\mathbb{E}[X] = \begin{cases} \sum_x x f(x), & X \textit{ diskret dengan pmf } f, \\ \int_{-\infty}^{\infty} x f(x)\, dx, & X \textit{ kontinu dengan pdf } f. \end{cases} \tag{2.7}$$

dengan syarat jumlahan atau integral tersebut konvergen. Ekspektasi $\mathbb{E}[X]$ juga biasa dinotasikan dengan μ atau μ_X.

Untuk fungsi dari suatu variabel random, ekspektasi dapat diperoleh dengan menggantikan x pada Persamaan (2.7) dengan fungsi tersebut.

Definisi 2.2.2 (Bain dan Engelhardt, 1992) *Jika u merupakan fungsi bernilai real dari variabel random X, maka*

$$\mathbb{E}[u(X)] = \begin{cases} \sum_x u(x) f(x), & X \textit{ diskret}, \\ \int_{-\infty}^{\infty} u(x) f(x)\, dx, & X \textit{ kontinu}. \end{cases} \tag{2.8}$$

Salah satu sifat penting dari ekspektasi adalah sifat linearitas. Jika terdapat dua fungsi dari variabel random yang dikalikan dengan suatu konstanta, maka ekspektasinya dapat dihitung secara terpisah.

2.2.2. Ekspektasi Vektor Random

Ekspektasi suatu vektor random diperoleh dengan menghitung ekspektasi dari setiap komponennya. Setiap komponen pada vektor ekspektasi menyatakan nilai harapan dari komponen vektor random yang bersesuaian. Sehingga untuk vektor random $\boldsymbol{X}$, Ekspektasinya adalah

$$\mathbb{E}[\boldsymbol{X}] = \big(\mathbb{E}[X_1], \ldots, \mathbb{E}[X_p]\big)^\top \in \mathbb{R}^p. \tag{2.9}$$

Jika $\boldsymbol{u}(X) = (u_1(X), \ldots, u_p(X))^\top$, maka ekspektasi juga dapat diterapkan pada setiap komponen sehingga

$$\mathbb{E}[\boldsymbol{u}(X)] = \begin{cases} \sum\limits_{x} \boldsymbol{u}(x) f(x), & X \text{ diskret}, \\ \int\limits_{-\infty}^{\infty} \boldsymbol{u}(x) f(x)\, dx, & X \text{ kontinu}. \end{cases} \tag{2.10}$$

Hasil dari Persamaan (2.10) merupakan sebuah vektor yang berada di $\mathbb{R}^p$.

2.2.3. Variansi dan Kovariansi

Variansi mengukur seberapa besar penyebaran nilai suatu variabel random terhadap nilai harapannya. Variansi yang kecil menunjukkan bahwa nilai variabel cenderung berada dekat dengan nilai harapannya, sedangkan variansi yang besar menunjukkan penyebaran yang lebih luas.

Definisi 2.2.3 (Bain dan Engelhardt, 1992) *Variansi variabel random X didefinisikan sebagai*

$$\mathrm{Var}(X) = \mathbb{E}\left[(X - \mathbb{E}[X])^2\right]. \tag{2.11}$$

Variansi juga dapat dinotasikan dengan σ^2 atau σ_X^2. Akar kuadrat dari variansi disebut standar deviasi, yaitu

$$\sigma = \sqrt{\mathrm{Var}(X)}. \tag{2.12}$$

Kovariansi mengukur kecenderungan dua variabel random berubah bersama. Kovariansi positif menunjukkan kecenderungan perubahan searah, sedangkan kovariansi negatif menunjukkan kecenderungan perubahan berlawanan arah. Kovariansi antara X dan Y didefinisikan sebagai

$$\mathrm{Cov}(X,Y) = \mathbb{E}\left[(X - \mathbb{E}[X])(Y - \mathbb{E}[Y])\right]. \tag{2.13}$$

Variansi merupakan kasus khusus kovariansi ketika kedua variabel yang dibandingkan adalah variabel yang sama, yaitu $\mathrm{Cov}(X,X) = \mathrm{Var}(X)$.

2.2.4. Koefisien Korelasi

Kovariansi menunjukkan arah hubungan antara dua variabel random, tetapi nilainya dipengaruhi oleh skala masing-masing variabel. Oleh karena itu, hubungan tersebut dapat dinyatakan dalam bentuk yang telah dinormalisasi menggunakan koefisien korelasi.

Definisi 2.2.4 (Bain dan Engelhardt, 1992) *Jika* $\mathrm{Var}(X) > 0$ *dan* $\mathrm{Var}(Y) > 0$*, maka koefisien korelasi antara* X *dan* Y *didefinisikan sebagai*

$$\rho_{XY} = \frac{\mathrm{Cov}(X,Y)}{\sqrt{\mathrm{Var}(X)\,\mathrm{Var}(Y)}}, \qquad -1 \le \rho_{XY} \le 1. \tag{2.14}$$

Nilai ρ_{XY} berada antara -1 dan 1. Nilai positif menunjukkan kecenderungan hubungan searah, sedangkan nilai negatif menunjukkan kecenderungan hubungan berlawanan arah. Jika nilainya mendekati nol, hubungan linear antara X dan Y semakin lemah.

2.3. Matriks

2.3.1. Jenis dan Operasi Matriks

Definisi 2.3.1 (Anton dan Rorres, 2010) *Matriks berordo* $m \times n$ *adalah susunan bilangan real dalam* m *baris dan* n *kolom, yang dinotasikan sebagai*

$$A = [a_{ij}] \in \mathbb{R}^{m \times n}, \tag{2.15}$$

dengan a_{ij} menyatakan elemen pada baris ke-i dan kolom ke-j.

Matriks bujur sangkar memiliki jumlah baris dan kolom yang sama. Matriks identitas I memiliki elemen diagonal utama bernilai satu dan elemen lainnya bernilai nol. Matriks diagonal memiliki elemen bernilai nol di luar diagonal utama, sedangkan matriks simetris memenuhi $A^\top = A$.

Operasi dasar matriks meliputi penjumlahan, perkalian dengan skalar, dan perkalian antarmatriks. Untuk matriks $A = [a_{ij}]$ dan $B = [b_{ij}]$ yang berordo sama, serta skalar $c \in \mathbb{R}$, berlaku

$$(A + B)_{ij} = a_{ij} + b_{ij}, \tag{2.16}$$

$$(cA)_{ij} = ca_{ij}. \tag{2.17}$$

Jika $A \in \mathbb{R}^{m\times n}$ dan $C \in \mathbb{R}^{n\times p}$, perkalian matriks didefinisikan sebagai

$$(AC)_{ik} = \sum_{j=1}^{n} a_{ij}c_{jk}. \tag{2.18}$$

Penjumlahan memerlukan kedua matriks berordo sama, sedangkan perkalian memerlukan jumlah kolom matriks pertama sama dengan jumlah baris matriks kedua.

Definisi 2.3.2 (Anton dan Rorres, 2010) *Trace dari matriks bujur sangkar $A \in \mathbb{R}^{n\times n}$ adalah jumlah elemen pada diagonal utamanya, yaitu*

$$\operatorname{tr}(A) = \sum_{i=1}^{n} a_{ii}. \tag{2.19}$$

Trace menghasilkan skalar dan hanya didefinisikan di sini untuk matriks bujur sangkar.

Definisi 2.3.3 (Anton dan Rorres, 2010) *Perkalian Hadamard dua matriks $A, B \in \mathbb{R}^{m\times n}$ adalah perkalian elemen demi elemen yang dinotasikan dengan $\odot$, dengan*

$$A \odot B = [a_{ij}b_{ij}] \in \mathbb{R}^{m\times n}. \tag{2.20}$$

Perkalian ini mensyaratkan kedua matriks memiliki ordo yang sama dan berbeda dari perkalian matriks biasa.

Definisi 2.3.4 (Anton dan Rorres, 2010) *Norma Frobenius kuadrat dari matriks $A \in \mathbb{R}^{m\times n}$ didefinisikan sebagai jumlah kuadrat seluruh elemennya,*

$$\|A\|_F^2 = \sum_{i=1}^{m}\sum_{j=1}^{n} a_{ij}^2 = \operatorname{tr}(A^\top A). \tag{2.21}$$

Nilai ini merupakan skalar taknegatif yang bernilai nol jika dan hanya jika A merupakan matriks nol.

2.4. Vektor

Definisi 2.4.1 (Anton dan Rorres, 2010) *Vektor berdimensi n adalah unsur $\mathbb{R}^n$, dituliskan sebagai matriks kolom*

$$x = (x_1, \ldots, x_n)^\top \in \mathbb{R}^n. \tag{2.22}$$

2.4.1. Hasil Kali Dalam dan Norma Euklides

Definisi 2.4.2 (Anton dan Rorres, 2010) *Hasil kali dalam (dot product) dua vektor $x, y \in \mathbb{R}^n$ adalah skalar*

$$\langle x, y\rangle = x^\top y = \sum_{i=1}^{n} x_i y_i. \tag{2.23}$$

Definisi 2.4.3 (Anton dan Rorres, 2010) *Norma Euklides vektor $x \in \mathbb{R}^n$ adalah*

$$\|x\|_2 = \sqrt{\langle x, x\rangle} = \left(\sum_{i=1}^{n} x_i^2\right)^{1/2}. \tag{2.24}$$

Norma ini mengukur panjang vektor dan bernilai nol jika dan hanya jika x merupakan vektor nol.

Definisi 2.4.4 (Ketaksamaan Cauchy–Schwarz; Anton dan Rorres, 2010) *Untuk se-*

tiap $x, y \in \mathbb{R}^n$ *berlaku*

$$|\langle x, y\rangle| \leq \|x\|_2 \|y\|_2. \tag{2.25}$$

Akibatnya, untuk vektor taknol, nilai hasil kali dalam yang telah dinormalisasi oleh panjang kedua vektor berada di antara -1 *dan* 1.

2.4.2. Kosinus Sudut Antar-Vektor

Definisi 2.4.5 (Anton dan Rorres, 2010) *Untuk vektor taknol* $x, y \in \mathbb{R}^n$*, kosinus sudut antara keduanya didefinisikan sebagai*

$$\cos(x, y) = \frac{\langle x, y\rangle}{\|x\|_2 \|y\|_2} = \frac{\displaystyle\sum_{i=1}^{n} x_i y_i}{\left(\displaystyle\sum_{i=1}^{n} x_i^2\right)^{1/2} \left(\displaystyle\sum_{i=1}^{n} y_i^2\right)^{1/2}}. \tag{2.26}$$

Berdasarkan Definisi 2.4.4, $-1 \leq \cos(x, y) \leq 1$. Nilai 1 menunjukkan bahwa kedua vektor searah, nilai -1 menunjukkan bahwa keduanya berlawanan arah, dan nilai 0 menunjukkan bahwa hasil kali dalamnya nol sehingga keduanya ortogonal. Dalam penelitian ini, kosinus digunakan untuk mengukur kemiripan arah antara representasi vektor.

2.5. Tensor

Definisi 2.5.1 *Tensor adalah susunan elemen yang diindeks oleh satu atau lebih indeks. Skalar merupakan tensor orde nol, vektor merupakan tensor orde satu, dan matriks merupakan tensor orde dua. Secara umum, tensor berordo* $(d_1, \ldots, d_k)$ *dapat dinotasikan sebagai*

$$\mathcal{X} \in \mathbb{R}^{d_1 \times d_2 \times \cdots \times d_k}, \tag{2.27}$$

dengan k *menyatakan orde tensor dan* d_j *menyatakan ukuran dimensi ke-*j.

Definisi 2.5.2 (Konkatenasi Tensor; PyTorch Contributors, 2026) *Konkatenasi (concatenation) adalah operasi menggabungkan dua tensor sepanjang dimensi tertentu tanpa menjumlahkan elemen-elemennya. Misalkan tensor* X *dan* Y *memiliki ukur-*

an yang sama pada seluruh dimensi selain dimensi ke-r. Konkatenasi keduanya pada dimensi tersebut dinotasikan sebagai

$$Z = \text{concat}_r(X, Y). \tag{2.28}$$

Ukuran tensor hasil pada dimensi ke-r merupakan jumlah ukuran kedua tensor pada dimensi tersebut, sedangkan ukuran dimensi lainnya tetap.

Sebagai ilustrasi, untuk

$$X = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}, \qquad Y = \begin{bmatrix} 5 \\ 6 \end{bmatrix},$$

konkatenasi secara horizontal menghasilkan

$$\text{concat}_2(X, Y) = \begin{bmatrix} 1 & 2 & 5 \\ 3 & 4 & 6 \end{bmatrix}.$$

Konkatenasi mempertahankan nilai dan urutan relatif elemen tensor asal. Operasi ini mensyaratkan ukuran kedua tensor sama pada dimensi selain dimensi yang digabungkan. Berbeda dari konkatenasi, penjumlahan tensor menggabungkan nilai elemen yang bersesuaian dan mensyaratkan ukuran kedua tensor sama.

2.6. Distribusi

2.6.1. Distribusi Multinomial

Distribusi multinomial memodelkan peluang kemunculan setiap kategori dalam sejumlah percobaan independen. Misalkan $Y_1, \ldots, Y_n$ merupakan variabel random independen dan berdistribusi identik secara kategorik dengan parameter $\boldsymbol{p} = (p_1, \ldots, p_V)^\top$.

Definisi 2.6.1 (Bain dan Engelhardt, 1992) *Misalkan N_i menyatakan banyaknya*

kemunculan kategori ke-i dalam n percobaan, yaitu

$$N_i = \sum_{t=1}^{n} \mathbb{I}(Y_t = i), \qquad i = 1, \ldots, V, \tag{2.29}$$

dengan $\mathbb{I}(Y_t = i)$ bernilai satu jika $Y_t = i$ dan nol selainnya. Vektor peluang $\boldsymbol{N} = (N_1, \ldots, N_V)^\top$ dikatakan berdistribusi multinomial dengan parameter n dan $\boldsymbol{p}$, ditulis

$$\boldsymbol{N} \sim \text{Multinomial}(n, \boldsymbol{p}),$$

jika fungsi massa probabilitasnya adalah

$$P(\boldsymbol{N} = \boldsymbol{c}) = \frac{n!}{\prod_{i=1}^{V} c_i!} \prod_{i=1}^{V} p_i^{c_i}, \tag{2.30}$$

untuk $c_i \in \{0, 1, 2, \ldots\}$ dengan $\sum_{i=1}^{V} c_i = n$, serta $p_i \geq 0$ dan $\sum_{i=1}^{V} p_i = 1$.

Pada Persamaan (2.30), c_i menyatakan peluang pengamatan pada kategori ke-i, sedangkan p_i merupakan probabilitas kategori tersebut pada setiap percobaan. Faktor $n! / \prod_{i=1}^{V} c_i!$ menyatakan banyaknya susunan hasil percobaan yang menghasilkan vektor peluang $\boldsymbol{c}$ yang sama.

Dalam inferensi Bayes, apabila parameter probabilitas $\boldsymbol{p}$ tidak diketahui dan diperlakukan sebagai parameter random $\boldsymbol{\theta}$, fungsi massa probabilitas multinomial menjadi likelihood bagi $\boldsymbol{\theta}$, yaitu

$$f(\boldsymbol{c} \mid \boldsymbol{\theta}) = \frac{n!}{\prod_{i=1}^{V} c_i!} \prod_{i=1}^{V} \theta_i^{c_i}. \tag{2.31}$$

Bentuk likelihood tersebut digunakan untuk memperoleh distribusi posterior ketika distribusi prior bagi $\boldsymbol{\theta}$ mengikuti distribusi Dirichlet.

2.6.2. Distribusi Kategorik dan Vektor Indikator

Distribusi kategorik dapat dipandang sebagai kasus khusus distribusi multinomial dengan satu percobaan. Misalkan variabel random Y mengambil nilai dalam

himpunan $\{1, \ldots, V\}$.

Definisi 2.6.2 (Casella dan Berger, 2002) *Variabel random Y dikatakan berdistribusi kategorik dengan parameter $\boldsymbol{p} = (p_1, \ldots, p_V)^\top$ jika fungsi massa probabilitasnya memenuhi*

$$P(Y = i) = p_i, \qquad i = 1, \ldots, V, \tag{2.32}$$

dengan $p_i \geq 0$ untuk setiap i dan $\sum_{i=1}^{V} p_i = 1$.

Setiap kategori i dapat direpresentasikan menggunakan vektor indikator atau vektor *one-hot* $\boldsymbol{e}_i \in \{0, 1\}^V$, yang didefinisikan sebagai

Definisi 2.6.3 *Vektor indikator $\boldsymbol{e}_i$ untuk kategori ke-i merupakan vektor berdimensi V yang komponen ke-i-nya bernilai satu dan seluruh komponen lainnya bernilai nol. Secara matematis,*

$$(\boldsymbol{e}_i)_j = \begin{cases} 1, & j = i, \\ 0, & j \neq i. \end{cases} \tag{2.33}$$

Setiap vektor indikator memiliki tepat satu komponen bernilai satu, sedangkan komponen lainnya bernilai nol. Jika Y merupakan variabel random dengan distribusi kategorik, maka $\boldsymbol{e}_Y$ menyatakan vektor indikator untuk kategori yang terpilih, sedangkan vektor $\boldsymbol{p}$ menyatakan probabilitas untuk seluruh kategori.

2.6.3. Distribusi Beta

Distribusi Beta merupakan distribusi kontinu yang digunakan untuk memodelkan variabel random pada interval $(0, 1)$. Distribusi ini memiliki dua parameter, yaitu $a, b > 0$. Fungsi Gamma dan fungsi Beta digunakan untuk menyatakan konstanta normalisasi distribusi tersebut [Casella dan Berger, 2002].

Definisi 2.6.4 (Casella dan Berger, 2002) *Fungsi Gamma didefinisikan sebagai*

$$\Gamma(a) = \int_0^\infty t^{a-1} e^{-t}\, dt, \qquad a > 0. \tag{2.34}$$

Fungsi ini memenuhi sifat rekursif

$$\Gamma(a+1) = a\Gamma(a). \tag{2.35}$$

Untuk bilangan bulat positif n, berlaku $\Gamma(n) = (n-1)!$.

Definisi 2.6.5 (Casella dan Berger, 2002) *Fungsi Beta didefinisikan sebagai*

$$B(a,b) = \frac{\Gamma(a)\Gamma(b)}{\Gamma(a+b)}, \qquad a,b > 0. \tag{2.36}$$

Fungsi ini juga memiliki representasi integral

$$B(a,b) = \int_0^1 x^{a-1}(1-x)^{b-1}\,dx. \tag{2.37}$$

Definisi 2.6.6 (Bain dan Engelhardt, 1992) *Variabel random kontinu X berdistribusi Beta dengan parameter $a, b > 0$, ditulis $X \sim \mathrm{Beta}(a,b)$, apabila fungsi kepadatan probabilitasnya adalah*

$$f(x;a,b) = \frac{x^{a-1}(1-x)^{b-1}}{B(a,b)}, \qquad 0 < x < 1. \tag{2.38}$$

Ekspektasi distribusi Beta diberikan oleh

$$\mathbb{E}[X] = \frac{a}{a+b}. \tag{2.39}$$

2.6.4. Distribusi Dirichlet

Distribusi Dirichlet merupakan perluasan distribusi Beta ke vektor probabilitas dengan lebih dari dua komponen. Distribusi ini digunakan untuk memodelkan vektor random yang berada pada simpleks probabilitas [Gelman dkk, 2013].

Definisi 2.6.7 (Gelman dkk, 2013) *Untuk parameter $\alpha = (\alpha_1, \ldots, \alpha_V)$ dengan $\alpha_i > 0$, definisikan $\alpha_0 = \sum_{i=1}^{V} \alpha_i$. Fungsi Beta multivariat adalah*

$$B(\alpha) = \frac{\prod_{i=1}^{V} \Gamma(\alpha_i)}{\Gamma(\alpha_0)}. \tag{2.40}$$

Definisi 2.6.8 (Gelman dkk, 2013) *Vektor random* $\theta = (\theta_1, \ldots, \theta_V)^\top$ *berdistribusi Dirichlet dengan parameter konsentrasi* $\alpha = (\alpha_1, \ldots, \alpha_V)$*, ditulis* $\theta \sim \mathrm{Dir}(\alpha)$*, apabila* θ *berada pada simpleks probabilitas* Δ^{V-1} *dan memiliki fungsi kepadatan*

$$f(\theta; \alpha) = \frac{1}{B(\alpha)} \prod_{i=1}^{V} \theta_i^{\alpha_i - 1}, \tag{2.41}$$

dengan $\theta_i \geq 0$ *dan* $\sum_{i=1}^{V} \theta_i = 1$*. Ekspektasi setiap komponennya adalah*

$$\mathbb{E}[\theta_i] = \frac{\alpha_i}{\alpha_0}, \qquad i = 1, \ldots, V. \tag{2.42}$$

Parameter α_i menentukan bobot relatif komponen ke-i, sedangkan α_0 merupakan jumlah seluruh parameter konsentrasi. Ekspektasi tersebut menunjukkan bahwa nilai rata-rata setiap komponen sebanding dengan parameternya terhadap total konsentrasi.

Dalam penelitian ini, sifat distribusi Dirichlet yang relevan adalah bentuk kepadatan dan ekspektasi komponennya. Bentuk kepadatan menjelaskan distribusi prior dan posterior, sedangkan ekspektasi digunakan untuk memperoleh bobot campuran dari parameter posterior.

2.7. Himpunan Konveks

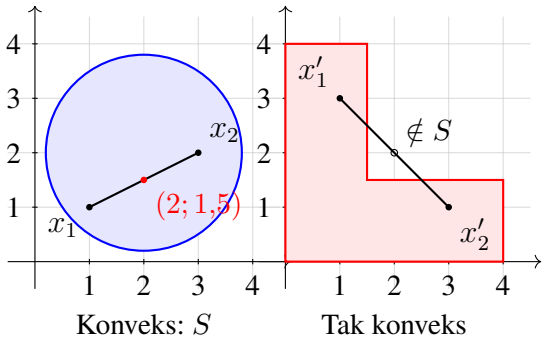
2.7.1. Himpunan Konveks

Sebuah himpunan disebut *konveks* apabila ruas garis yang menghubungkan sembarang dua titik di dalamnya seluruhnya berada di dalam himpunan tersebut. Secara matematis, misalkan $S \subseteq \mathbb{R}^n$ dan $x, y \in S$. Titik-titik pada ruas garis yang menghubungkan x dan y dapat dinyatakan sebagai $\theta x + (1-\theta)y$, dengan $\theta \in [0, 1]$. Nilai $\theta = 0$ menghasilkan y, sedangkan $\theta = 1$ menghasilkan x.

Definisi 2.7.1 (Boyd dan Vandenberghe, 2004) *Himpunan* $S \subseteq \mathbb{R}^n$ *disebut konveks apabila untuk setiap* $x, y \in S$ *dan* $\theta \in [0, 1]$ *berlaku*

$$\theta x + (1 - \theta)y \in S. \tag{2.43}$$

Gambar 2.1 memperlihatkan contoh himpunan konveks dan tak konveks. Pada himpunan konveks, ruas garis yang menghubungkan dua titik di dalamnya tetap berada di dalam himpunan. Sebaliknya, pada himpunan tak konveks, sebagian ruas garis tersebut dapat berada di luar himpunan.



Gambar 2.1 Contoh himpunan konveks (kiri) dan tak konveks (kanan). Pada himpunan tak konveks, ruas garis antara x'_1 dan x'_2 melewati titik $(2, 2)$ yang berada di luar himpunan.

2.7.2. Kombinasi Konveks dan Convex Hull

Konsep himpunan konveks dapat diperluas untuk mencampurkan lebih dari dua titik. Campuran ini disebut *kombinasi konveks* apabila setiap bobot tidak negatif dan jumlah seluruh bobotnya sama dengan satu.

Definisi 2.7.2 (Boyd dan Vandenberghe, 2004) *Untuk titik-titik $x_1, \ldots, x_k \in \mathbb{R}^n$ dan skalar $\theta_1, \ldots, \theta_k$ yang memenuhi $\theta_i \geq 0$ dan $\sum_{i=1}^{k} \theta_i = 1$, titik*

$$\sum_{i=1}^{k} \theta_i x_i \tag{2.44}$$

disebut kombinasi konveks dari $x_1, \ldots, x_k$.

Kumpulan seluruh kombinasi konveks dari titik-titik dalam suatu himpunan membentuk *convex hull*, yaitu himpunan konveks terkecil yang memuat himpunan tersebut.

Definisi 2.7.3 (Boyd dan Vandenberghe, 2004) *Convex hull dari himpunan $S \subseteq \mathbb{R}^n$,*

ditulis $\mathrm{conv}(S)$*, didefinisikan sebagai*

$$\mathrm{conv}(S) = \left\{ \sum_{i=1}^{k} \theta_i x_i \;\middle|\; \begin{array}{l} k \in \mathbb{N},\ x_i \in S,\ \theta_i \geq 0, \\ \displaystyle\sum_{i=1}^{k} \theta_i = 1 \end{array} \right\}, \tag{2.45}$$

sehingga $\mathrm{conv}(S)$ *merupakan himpunan konveks terkecil yang memuat* S*.*

2.7.3. Simpleks Probabilitas

Salah satu contoh penting yang berkaitan dengan kombinasi konveks adalah *simpleks probabilitas*, yaitu himpunan seluruh vektor probabilitas untuk V kategori. Setiap komponennya tidak negatif dan jumlah seluruh komponennya sama dengan satu.

Definisi 2.7.4 (Boyd dan Vandenberghe, 2004) *Simpleks probabilitas berdimensi* $V - 1$ *didefinisikan sebagai*

$$\Delta^{V-1} = \left\{ p = (p_1, \ldots, p_V)^\top \in \mathbb{R}^V \;\middle|\; p_i \geq 0,\ \sum_{i=1}^{V} p_i = 1 \right\}. \tag{2.46}$$

Simpleks probabilitas merupakan *convex hull* dari vektor indikator $e_1, \ldots, e_V$ (Definisi 2.6.3), yaitu

$$\Delta^{V-1} = \mathrm{conv}\{e_1, \ldots, e_V\}. \tag{2.47}$$

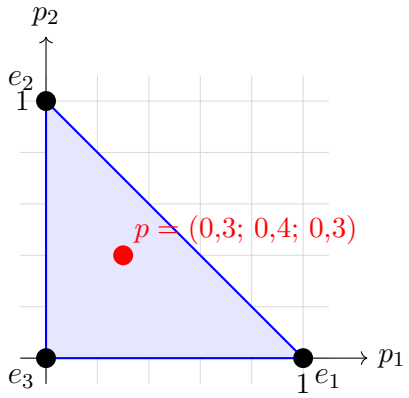
Setiap vektor probabilitas $p \in \Delta^{V-1}$ dapat dinyatakan sebagai kombinasi konveks

$$p = \sum_{i=1}^{V} p_i e_i. \tag{2.48}$$

Dalam representasi ini, setiap vektor indikator merupakan titik sudut simpleks yang menyatakan probabilitas satu pada satu kategori, sedangkan titik-titik lainnya menyatakan distribusi probabilitas yang tersebar di antara beberapa kategori.

Gambar 2.2 mengilustrasikan simpleks probabilitas untuk $V = 3$ kategori. Karena $p_1 + p_2 + p_3 = 1$, simpleks tersebut dapat digambarkan pada bidang (p_1, p_2)

dengan $p_3 = 1 - p_1 - p_2$.



Gambar 2.2 Simpleks probabilitas Δ^2 untuk $V = 3$ kategori pada bidang (p_1, p_2). Titik sudut e_1, e_2, e_3 menyatakan distribusi yang pasti pada satu kategori, sedangkan titik di bagian dalam menyatakan distribusi campuran.

2.8. Entropi

2.8.1. Entropi Shannon

Entropi Shannon mengukur ketidakpastian suatu variabel random diskret. Misalkan X merupakan variabel random diskret dengan ruang nilai $\mathcal{X}$ dan fungsi massa probabilitas $p(x)$. Entropi Shannon didefinisikan sebagai berikut.

Definisi 2.8.1 (Cover dan Thomas, 2006) *Entropi Shannon dari variabel random diskret X adalah*

$$H(X) = -\sum_{x \in \mathcal{X}} p(x) \log p(x), \tag{2.49}$$

dengan konvensi $0 \log 0 := 0$.

2.8.2. Batas Entropi

Untuk variabel random diskret X dengan ruang nilai berhingga $\mathcal{X}$, entropi Shannon memenuhi batas berikut [Cover dan Thomas, 2006].

$$0 \leq H(X) \leq \log |\mathcal{X}|. \tag{2.50}$$

Batas bawah dicapai jika dan hanya jika distribusi probabilitas X terkonsentrasi pada satu nilai, sedangkan batas atas dicapai jika dan hanya jika distribusi

probabilitas X seragam pada $\mathcal{X}$.

2.9. Inferensi Bayes

Inferensi Bayes merupakan pendekatan untuk melakukan inferensi terhadap parameter yang tidak diketahui dengan menggabungkan informasi awal mengenai parameter dan informasi yang diperoleh dari data. Informasi awal dinyatakan melalui distribusi prior, sedangkan informasi dari data dinyatakan melalui likelihood. Penggabungan keduanya menghasilkan distribusi posterior yang menggambarkan ketidakpastian parameter setelah data diamati [Casella dan Berger, 2002].

2.9.1. Definisi Bayes: Prior, Likelihood, dan Posterior

Definisi 2.9.1 (Rumus Bayes; Casella dan Berger, 2002) *Misalkan θ merupakan parameter dan Y merupakan variabel random yang menghasilkan data y. Jika $f(\theta)$ merupakan distribusi prior dan $f(y \mid \theta)$ merupakan fungsi massa peluang atau fungsi kepadatan bersyarat dari Y diberikan θ, maka distribusi posterior θ setelah data y diamati adalah*

$$f(\theta \mid y) = \frac{f(y \mid \theta) f(\theta)}{f(y)}. \tag{2.51}$$

Untuk parameter kontinu, evidence $f(y)$ diperoleh melalui

$$f(y) = \int f(y \mid \theta) f(\theta)\, d\theta. \tag{2.52}$$

Untuk parameter diskret, integral digantikan dengan penjumlahan.

Pada Persamaan (2.51), $f(\theta)$ merupakan prior, $f(y \mid \theta)$ merupakan likelihood, $f(y)$ merupakan evidence, dan $f(\theta \mid y)$ merupakan posterior. Evidence berfungsi sebagai konstanta normalisasi agar posterior merupakan distribusi probabilitas yang sah. Karena evidence tidak bergantung pada θ, posterior juga dapat dinyatakan secara proporsional sebagai

$$f(\theta \mid y) \propto f(y \mid \theta) f(\theta). \tag{2.53}$$

Bentuk proporsional ini memudahkan penentuan posterior ketika hasil perkalian likelihood dan prior sudah dapat dikenali sebagai kernel suatu keluarga distribusi.

2.9.2. Prior Konjugat

Definisi 2.9.2 (Casella dan Berger, 2002) *Keluarga distribusi prior $\mathcal{F}$ disebut konjugat terhadap likelihood $f(y \mid \theta)$ apabila posterior yang diperoleh dari setiap prior dalam $\mathcal{F}$ juga termasuk dalam keluarga distribusi $\mathcal{F}$.*

Kekonjugatan memungkinkan posterior dinyatakan dalam bentuk tertutup tanpa harus menghitung evidence secara eksplisit. Salah satu contoh yang relevan dalam penelitian ini adalah kekonjugatan Dirichlet–Multinomial.

2.9.3. Kekonjugatan Dirichlet–Multinomial

Definisi 2.9.3 (Gelman dkk, 2013) *Misalkan vektor probabilitas $\theta = (\theta_1, \ldots, \theta_V)^\top$ memiliki prior*

$$\theta \sim \mathrm{Dir}(\alpha), \qquad \alpha = (\alpha_1, \ldots, \alpha_V)^\top,$$

dengan $\alpha_i > 0$. Jika, diberikan θ, vektor frekuensi kategori $\boldsymbol{c} = (c_1, \ldots, c_V)^\top$ mengikuti distribusi Multinomial dengan ukuran sampel n, maka posteriornya adalah

$$\theta \mid \boldsymbol{c} \sim \mathrm{Dir}(\alpha + \boldsymbol{c}). \tag{2.54}$$

Hasil tersebut dapat diturunkan menggunakan Rumus Bayes dengan mengalikan likelihood Multinomial dan prior Dirichlet. Berdasarkan Persamaan (2.30), fungsi likelihood untuk vektor frekuensi $\boldsymbol{c}$ adalah

$$f(\boldsymbol{c} \mid \theta) = \frac{n!}{\prod_{i=1}^{V} c_i!} \prod_{i=1}^{V} \theta_i^{c_i}, \tag{2.55}$$

dengan $\sum_{i=1}^{V} c_i = n$. Sementara itu, berdasarkan Persamaan (2.41), fungsi kepa-

datan prior Dirichlet adalah

$$f(\theta) = \frac{1}{B(\alpha)} \prod_{i=1}^{V} \theta_i^{\alpha_i - 1}, \tag{2.56}$$

dengan $B(\alpha)$ merupakan konstanta normalisasi distribusi Dirichlet.

Menurut Rumus Bayes pada Persamaan (2.51), posterior sebanding dengan perkalian likelihood dan prior. Oleh karena itu, diperoleh

$$\begin{aligned} f(\theta \mid \boldsymbol{c}) &\propto f(\boldsymbol{c} \mid \theta) f(\theta) \\ &= \frac{n!}{\prod_{i=1}^{V} c_i!} \frac{1}{B(\alpha)} \left(\prod_{i=1}^{V} \theta_i^{c_i} \right) \left(\prod_{i=1}^{V} \theta_i^{\alpha_i - 1} \right). \end{aligned} \tag{2.57}$$

Pada Persamaan (2.57), faktor $\frac{n!}{\prod_{i=1}^{V} c_i!}$ dan $\frac{1}{B(\alpha)}$ tidak bergantung pada θ. Karena itu, kedua faktor tersebut dapat diserap ke dalam konstanta proporsionalitas. Selanjutnya, menggunakan sifat perkalian berpangkat $x^a x^b = x^{a+b}$, diperoleh

$$\begin{aligned} f(\theta \mid \boldsymbol{c}) &\propto \prod_{i=1}^{V} \theta_i^{c_i} \theta_i^{\alpha_i - 1} \\ &= \prod_{i=1}^{V} \theta_i^{\alpha_i + c_i - 1}. \end{aligned} \tag{2.58}$$

Bentuk pada Persamaan (2.58) sama dengan bentuk kernel distribusi Dirichlet, yaitu hasil kali setiap komponen probabilitas yang dipangkatkan dengan parameter distribusinya dikurangi satu. Dengan membandingkannya dengan fungsi kepadatan Dirichlet, parameter posterior dapat diidentifikasi sebagai $\alpha_i + c_i$ untuk setiap kategori i.

Agar menjadi fungsi kepadatan probabilitas, kernel tersebut harus dinormalisasi. Konstanta normalisasinya adalah $B(\alpha + \boldsymbol{c})$, sehingga posterior lengkapnya menjadi

$$f(\theta \mid \boldsymbol{c}) = \frac{1}{B(\alpha + \boldsymbol{c})} \prod_{i=1}^{V} \theta_i^{\alpha_i + c_i - 1} \tag{2.59}$$

dan diperoleh $\theta \mid \boldsymbol{c} \sim \mathrm{Dir}(\alpha + \boldsymbol{c})$, sebagaimana dinyatakan pada Persamaan (2.54).

Hasil ini menunjukkan bahwa keluarga prior Dirichlet tetap menjadi keluarga posterior setelah pengamatan mengikuti likelihood Multinomial.

Secara intuitif, pembaruan posterior menambahkan frekuensi pengamatan c_i pada parameter prior α_i. Untuk satu pengamatan pada kategori y, vektor frekuensinya merupakan vektor indikator e_y, yang didefinisikan sebagai

$$(e_y)_i = \begin{cases} 1, & i = y, \\ 0, & i \neq y. \end{cases}$$

Oleh karena itu, pembaruan posterior untuk satu pengamatan adalah

$$\theta \mid y \sim \mathrm{Dir}(\alpha + e_y). \tag{2.60}$$

Artinya, parameter prior untuk kategori yang diamati bertambah satu, sedangkan parameter kategori lainnya tidak berubah.

Definisi 2.9.4 (Gelman dkk, 2013) *Parameter prior α_i disebut pseudo-count kategori i, sedangkan*

$$\alpha_0 = \sum_{i=1}^{V} \alpha_i$$

disebut massa konsentrasi prior. Pembaruan $\alpha_i \mapsto \alpha_i + c_i$ menambahkan frekuensi pengamatan kategori i pada pseudo-count kategori tersebut.

Berdasarkan rumus ekspektasi distribusi Dirichlet pada Persamaan (2.42), ekspektasi komponen posterior diperoleh dengan mengganti parameter prior α_i menjadi parameter posterior $\alpha_i + c_i$:

$$\begin{aligned} \mathbb{E}[\theta_i \mid \boldsymbol{c}] &= \frac{\alpha_i + c_i}{\sum_{j=1}^{V}(\alpha_j + c_j)} \\ &= \frac{\alpha_i + c_i}{\alpha_0 + n}, \qquad i = 1, \ldots, V, \end{aligned} \tag{2.61}$$

karena $\sum_{i=1}^{V} c_i = n$. Persamaan ini menunjukkan bahwa rerata posterior ditentukan oleh pseudo-count awal dan frekuensi pengamatan, sedangkan penyebutnya

merupakan total konsentrasi setelah pembaruan.

2.9.4. Rerata Posterior sebagai Estimator Bayes

Definisi 2.9.5 (Casella dan Berger, 2002) *Misalkan $\theta \in \mathbb{R}^V$ merupakan parameter random dan $\delta = \delta(y) \in \mathbb{R}^V$ merupakan penduga parameter berdasarkan data y. Risiko posterior dengan squared loss didefinisikan sebagai*

$$R(\delta \mid y) = \mathbb{E}\left[\|\theta - \delta\|_2^2 \mid y\right]. \tag{2.62}$$

Penduga yang meminimumkan risiko posterior tersebut disebut estimator Bayes di bawah squared loss.

Untuk menunjukkan bahwa rerata posterior merupakan estimator Bayes di bawah *squared loss*, definisikan $\mu_i = \mathbb{E}[\theta_i \mid y]$. Karena kuadrat norma Euclidean merupakan jumlah kuadrat setiap komponen, risiko posterior dapat dituliskan sebagai

$$\begin{aligned} R(\delta \mid y) &= \mathbb{E}\left[\sum_{i=1}^{V}(\theta_i - \delta_i)^2 \mid y\right] \\ &= \sum_{i=1}^{V}\mathbb{E}\left[(\theta_i - \delta_i)^2 \mid y\right]. \end{aligned} \tag{2.63}$$

Langkah kedua menggunakan linearitas ekspektasi, yaitu ekspektasi dari jumlah berhingga sama dengan jumlah ekspektasi masing-masing suku.

Selanjutnya, setiap selisih $\theta_i - \delta_i$ dapat diuraikan dengan menambahkan dan mengurangkan rerata posterior μ_i:

$$\theta_i - \delta_i = (\theta_i - \mu_i) + (\mu_i - \delta_i). \tag{2.64}$$

Dengan menguadratkan kedua ruas, diperoleh

$$(\theta_i - \delta_i)^2 = (\theta_i - \mu_i)^2 + 2(\theta_i - \mu_i)(\mu_i - \delta_i) + (\mu_i - \delta_i)^2. \tag{2.65}$$

Mengambil ekspektasi bersyarat pada data y menghasilkan

$$\begin{aligned} &\mathbb{E}\left[(\theta_i - \delta_i)^2 \mid y\right] \\ &= \mathbb{E}\left[(\theta_i - \mu_i)^2 \mid y\right] \\ &\quad + 2(\mu_i - \delta_i)\mathbb{E}\left[\theta_i - \mu_i \mid y\right] + (\mu_i - \delta_i)^2. \end{aligned} \tag{2.66}$$

Pada langkah ini, μ_i dan δ_i diperlakukan sebagai konstanta karena keduanya ditentukan oleh distribusi posterior setelah data y diamati.

Berdasarkan definisi rerata posterior,

$$\begin{aligned} \mathbb{E}\left[\theta_i - \mu_i \mid y\right] &= \mathbb{E}[\theta_i \mid y] - \mu_i \\ &= \mu_i - \mu_i = 0. \end{aligned} \tag{2.67}$$

Akibatnya, suku silang pada Persamaan (2.66) bernilai nol. Sementara itu, suku pertama merupakan varians posterior karena $\mu_i = \mathbb{E}[\theta_i \mid y]$. Dengan demikian,

$$\mathbb{E}\left[(\theta_i - \delta_i)^2 \mid y\right] = \mathrm{Var}(\theta_i \mid y) + (\delta_i - \mu_i)^2. \tag{2.68}$$

Substitusi Persamaan (2.68) ke dalam Persamaan (2.63) memberikan

$$R(\delta \mid y) = \sum_{i=1}^{V} \mathrm{Var}(\theta_i \mid y) + \sum_{i=1}^{V} (\delta_i - \mu_i)^2. \tag{2.69}$$

Suku pertama tidak bergantung pada penduga δ, sedangkan suku kedua merupakan jumlah kuadrat yang selalu tidak negatif. Suku kedua mencapai nilai minimum nol apabila $\delta_i = \mu_i$ untuk setiap i. Oleh karena itu, risiko posterior minimum dicapai

oleh

$$\delta^* = \mathbb{E}[\theta \mid y] = \begin{pmatrix} \mathbb{E}[\theta_1 \mid y] \\ \vdots \\ \mathbb{E}[\theta_V \mid y] \end{pmatrix}. \tag{2.70}$$

Rerata posterior merupakan estimator Bayes di bawah *squared loss*.

Dalam konteks prior Dirichlet dan likelihood Multinomial, Persamaan (2.61) memberikan bentuk eksplisit estimator Bayes tersebut:

$$\delta_i^* = \frac{\alpha_i + c_i}{\alpha_0 + n}, \qquad i = 1, \ldots, V. \tag{2.71}$$

Hasil ini menjadi landasan matematis untuk menggunakan rerata posterior sebagai bobot campuran dalam metode Mixture of Inputs.

2.10. Model Bahasa dan Arsitektur *Transformer*

2.10.1. Model Bahasa Autoregresif

Definisi 2.10.1 (Goodfellow dkk, 2016) *Misalkan $\mathcal{V} = \{1, \ldots, V\}$ kosakata (vocabulary) berukuran V, dan $x = (x_1, \ldots, x_n)$ barisan token dengan $x_t \in \mathcal{V}$. Model bahasa (language model) adalah distribusi peluang atas barisan token, difaktorkan menurut aturan rantai peluang sebagai*

$$p(x) = \prod_{t=1}^{n} p(x_t \mid x_{<t}), \tag{2.72}$$

dengan $x_{<t} = (x_1, \ldots, x_{t-1})$. Model yang memfaktorkan peluangnya demikian — token ke-t dibangkitkan bergantung pada seluruh token sebelumnya — disebut autoregresif.

Persamaan (2.72) adalah penerapan berulang aturan rantai peluang. Untuk $n = 3$ misalnya, produknya terurai menjadi tiga faktor,

$$p(x_1, x_2, x_3) = p(x_1)\, p(x_2 \mid x_1)\, p(x_3 \mid x_1, x_2), \tag{2.73}$$

dengan $p(x_1) = p(x_1 \mid x_{<1})$ memakai $x_{<1} = \varnothing$ (barisan kosong, tidak ada token sebelum token pertama). Setiap faktor pada Persamaan (2.73) adalah distribusi atas V kategori yang bersyarat pada token-token sebelumnya, sehingga membangkitkan barisan x berarti membangkitkan token satu per satu secara berurutan: x_1 terlebih dahulu, kemudian x_2 dengan x_1 diketahui, dan seterusnya.

2.10.2. Jaringan Saraf, Fungsi Aktivasi, dan Fungsi *Softmax*

Komponen dasar jaringan saraf tiruan adalah lapisan (*layer*) yang memetakan masukan $u \in \mathbb{R}^m$ menjadi keluaran $\sigma(Wu + b)$, dengan W dan b merupakan parameter yang dipelajari, sedangkan σ adalah fungsi aktivasi taklinear, seperti ReLU atau *tanh*. Fungsi aktivasi memungkinkan tumpukan lapisan memodelkan hubungan yang lebih kompleks daripada kombinasi linear semata [Goodfellow dkk, 2016]. Arsitektur *Transformer* juga tersusun atas rangkaian lapisan yang melibatkan operasi linear dan fungsi aktivasi.

Pada model bahasa, lapisan keluaran menghasilkan vektor *logit* $\ell = (\ell_1, \ldots, \ell_V)^\top \in \mathbb{R}^V$, dengan setiap ℓ_i merupakan skor untuk token ke-i dari kosakata berukuran V. Agar skor tersebut dapat digunakan sebagai distribusi probabilitas, diperlukan fungsi *softmax*.

Definisi 2.10.2 (Goodfellow dkk, 2016) *Fungsi softmax memetakan vektor logit $\ell \in \mathbb{R}^V$ menjadi vektor probabilitas $p = \mathrm{softmax}(\ell) \in \Delta^{V-1}$, dengan komponen*

$$p_i = \frac{\exp(\ell_i)}{\sum_{j=1}^{V} \exp(\ell_j)}, \qquad i = 1, \ldots, V. \tag{2.74}$$

Setiap komponen p_i menyatakan probabilitas kategori ke-i. Nilainya positif dan jumlah seluruh komponennya sama dengan satu, sehingga p merupakan distribusi probabilitas pada simpleks Δ^{V-1} (Definisi 2.7.4).

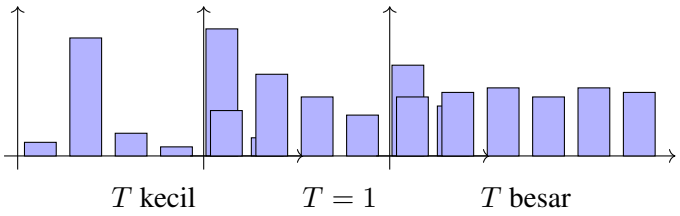
Dalam penerapannya, fungsi *softmax* dapat menggunakan parameter suhu

(*temperature*) $T > 0$ dengan mengganti logit ℓ_i menjadi ℓ_i/T:

$$p_i = \frac{\exp(\ell_i/T)}{\sum_{j=1}^{V} \exp(\ell_j/T)}. \tag{2.75}$$

Parameter T mengatur tingkat ketajaman distribusi. Ketika T mengecil, perbedaan antarlogit menjadi lebih berpengaruh sehingga probabilitas cenderung terkonsentrasi pada kategori dengan logit terbesar. Sebaliknya, ketika T membesar, perbedaan tersebut semakin berkurang dan distribusi mendekati seragam.

Gambar 2.3 mengilustrasikan pengaruh suhu terhadap bentuk distribusi.



Gambar 2.3 Ilustrasi pengaruh suhu T terhadap distribusi $p = \mathrm{softmax}(\ell/T)$ untuk logit yang sama. Suhu kecil menghasilkan distribusi yang lebih tajam, sedangkan suhu besar menghasilkan distribusi yang lebih mendekati seragam.

2.10.3. Mekanisme Atensi dan *KV-Cache*

Transformer memproses representasi token melalui mekanisme atensi (*attention*), yang memungkinkan representasi setiap posisi dibentuk dari kombinasi berbobot representasi posisi-posisi lain yang relevan.

Definisi 2.10.3 (Vaswani dkk, 2017) *Untuk matriks query (query) $Q \in \mathbb{R}^{n \times d_k}$, key (key) $K \in \mathbb{R}^{n \times d_k}$, dan nilai (value) $V \in \mathbb{R}^{n \times d_v}$, mekanisme atensi produk-titik berskala (scaled dot-product attention) didefinisikan sebagai*

$$\mathrm{Atensi}(Q, K, V) = \mathrm{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V, \tag{2.76}$$

dengan fungsi softmax diterapkan pada setiap baris matriks.

Perkalian matriks (*MatMul*). Operasi pertama menghitung $QK^\top$. Karena $Q \in \mathbb{R}^{n \times d_k}$ dan $K^\top \in \mathbb{R}^{d_k \times n}$, hasilnya merupakan matriks berukuran $n \times n$. Setiap

elemennya merupakan hasil kali titik (*dot product*) antara satu baris Q dan satu baris K:

$$(QK^\top)_{ij} = \sum_{r=1}^{d_k} Q_{ir} K_{jr}. \tag{2.77}$$

Nilai tersebut mengukur kesesuaian antara *query* pada posisi i dan *key* pada posisi j. Dengan demikian, matriks $QK^\top$ memuat skor kesesuaian setiap posisi terhadap seluruh posisi lainnya.

Penskalaan (*Scale*). Skor kesesuaian kemudian dibagi dengan $\sqrt{d_k}$:

$$S = \frac{QK^\top}{\sqrt{d_k}}. \tag{2.78}$$

Pembagian ini mengendalikan besarnya skor sebelum fungsi *softmax* diterapkan. Ketika dimensi d_k membesar, hasil kali titik cenderung memiliki varians yang lebih besar dengan asumsi komponen *query* dan *key* memiliki varians yang terkendali dan tidak berkorelasi. Skor yang terlalu besar dapat membuat distribusi *softmax* terlalu terkonsentrasi. Karena itu, faktor $\sqrt{d_k}$ membantu menjaga skala skor agar proses pembobotan lebih stabil.

Fungsi *softmax*. Fungsi *softmax* diterapkan pada setiap baris matriks S untuk mengubah skor kesesuaian menjadi bobot probabilitas:

$$A_{ij} = \frac{\exp(S_{ij})}{\sum_{r=1}^{n} \exp(S_{ir})}. \tag{2.79}$$

Setiap elemen A_{ij} menunjukkan bobot posisi j ketika representasi posisi i dibentuk. Untuk setiap baris i, berlaku $A_{ij} \geq 0$ dan $\sum_{j=1}^{n} A_{ij} = 1$. Dengan demikian, bobot tersebut dapat digunakan untuk menggabungkan informasi dari berbagai posisi.

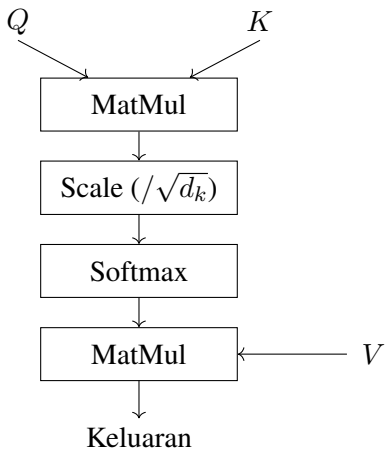
Pembobotan nilai (*MatMul*). Operasi terakhir mengalikan matriks bobot A dengan matriks *value* V:

$$O = AV. \tag{2.80}$$

Karena $A \in \mathbb{R}^{n \times n}$ dan $V \in \mathbb{R}^{n \times d_v}$, keluaran $O \in \mathbb{R}^{n \times d_v}$. Setiap baris keluaran merupakan kombinasi berbobot baris-baris V, dengan bobot yang ditentukan oleh skor kesesuaian pada tahap sebelumnya. Oleh karena itu, representasi keluaran menggabungkan informasi dari posisi-posisi yang memperoleh bobot atensi lebih besar.

Dalam praktiknya, operasi atensi dapat dilakukan pada beberapa *head* dengan proyeksi dan dimensi masing-masing, kemudian hasilnya digabungkan melalui mekanisme *multi-head attention*.

Gambar 2.4 mengilustrasikan urutan operasi tersebut.



Gambar 2.4 Alur perhitungan atensi produk-titik berskala pada Persamaan (2.76)**, mengikuti diagram Vaswani dkk [2017].**

Pada pembangkitan teks autoregresif, token dihasilkan satu per satu. Untuk setiap lapisan, representasi token yang telah diproses menghasilkan matriks *key* dan *value* yang dapat digunakan kembali pada langkah berikutnya. Tanpa penyimpanan, model perlu menghitung ulang *key* dan *value* token-token sebelumnya pada setiap langkah.

Definisi 2.10.4 *KV-cache pada langkah ke-t menyimpan matriks key dan value untuk token-token sebelumnya pada setiap lapisan l:*

$$\mathcal{M}_t^{(l)} = \left\{ K_{1:t-1}^{(l)}, V_{1:t-1}^{(l)} \right\}. \tag{2.81}$$

Pada langkah t, key dan value token baru dihitung lalu digabungkan dengan cache

yang telah tersimpan. Dengan demikian, model tidak perlu menghitung ulang key dan value seluruh token sebelumnya.

2.11. Strategi Pengambilan Sampel Token

Setelah distribusi probabilitas token $p = \mathrm{softmax}(\ell/T) \in \Delta^{V-1}$ diperoleh (Persamaan (2.74)), model perlu menentukan token berikutnya x_t berdasarkan distribusi tersebut. Salah satu cara adalah *greedy decoding*, yaitu memilih token dengan probabilitas terbesar,

$$x_t = \arg\max_{i \in \mathcal{V}} p_i. \tag{2.82}$$

Strategi ini bersifat deterministik untuk distribusi yang sama. Alternatifnya, model dapat mengambil sampel dari distribusi kategorik,

$$x_t \sim \mathrm{Kategorik}(p), \tag{2.83}$$

sehingga peluang terpilihnya setiap token sebanding dengan probabilitasnya. Berbeda dari *greedy decoding*, strategi ini memungkinkan token dengan probabilitas lebih rendah tetap terpilih. Namun, sampling langsung dari p juga memungkinkan token dengan probabilitas sangat kecil terpilih.

Untuk membatasi pilihan token, distribusi probabilitas dapat dipotong sebelum sampling dilakukan. Dua strategi yang umum digunakan adalah *top-k sampling* dan *nucleus sampling* (*top-p sampling*).

Definisi 2.11.1 (Holtzman dkk, 2020) *Untuk $k \in \{1, \ldots, V\}$, misalkan $\mathcal{I}_k \subseteq \mathcal{V}$ merupakan himpunan k token dengan probabilitas terbesar. Pada top-k sampling, probabilitas token di luar himpunan tersebut dibuat nol, sedangkan probabilitas token yang tersisa dinormalisasi ulang:*

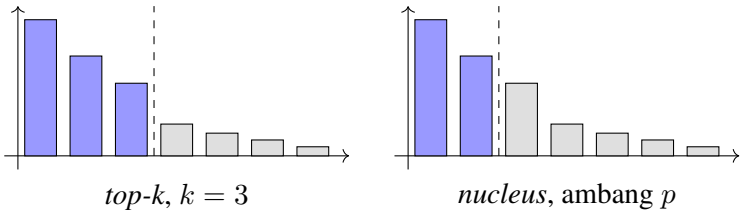
$$\widetilde{p}_i = \begin{cases} \dfrac{p_i}{\sum_{j \in \mathcal{I}_k} p_j}, & i \in \mathcal{I}_k, \\ 0, & i \notin \mathcal{I}_k. \end{cases} \tag{2.84}$$

Dengan demikian, hanya k token dengan probabilitas terbesar yang dapat dipilih.

Pada nucleus sampling atau top-p sampling, himpunan token yang dipertahankan ditentukan berdasarkan probabilitas kumulatif. Misalkan token diurutkan berdasarkan probabilitas yang menurun. Himpunan $\mathcal{I}_p$ merupakan himpunan terkecil yang memenuhi

$$\sum_{i \in \mathcal{I}_p} p_i \geq p, \qquad p \in (0, 1). \tag{2.85}$$

Sama seperti pada top-k sampling, probabilitas token di luar $\mathcal{I}_p$ dibuat nol dan probabilitas token yang tersisa dinormalisasi ulang sebelum sampling dilakukan. Perbedaannya, top-k sampling membatasi banyaknya token, sedangkan top-p sampling membatasi massa probabilitas kumulatif yang dipertahankan.



Gambar 2.5 Perbandingan *top-k sampling* (kiri, memotong pada banyaknya kategori) dan *nucleus sampling* (kanan, memotong pada peluang kumulatif). Batang biru menandai kategori yang tersisa untuk disampling ulang; batang abu-abu tersisih.

Gambar 2.5 mengilustrasikan perbedaan kedua strategi tersebut. Pada *top-k sampling*, jumlah token yang dipertahankan ditentukan oleh k, sedangkan pada *top-p sampling*, jumlahnya dapat berubah mengikuti bentuk distribusi probabilitas.

Dalam penelitian ini, strategi pengambilan token perlu dibedakan berdasarkan penggunaannya. Pada medium komunikasi teks (`text`), setiap agen menghasilkan keluaran teks melalui proses generasi autoregresif. Implementasi menggunakan sampling stokastik dengan parameter suhu $T = 0{,}6$, *top-k* sebesar 20, dan *top-p* sebesar 0,95 sebagai nilai bawaan mesin generasi. Pada setiap langkah, token berikutnya dipilih berdasarkan distribusi yang telah dipengaruhi oleh suhu dan pembatasan sampling tersebut. Proses ini diulang hingga generasi berakhir. Keluaran teks dari agen perantara kemudian diberikan kepada agen berikutnya sebagai konteks tekstual. Agen *judger* juga menggunakan mekanisme generasi teks ini un-

tuk menghasilkan jawaban akhir.

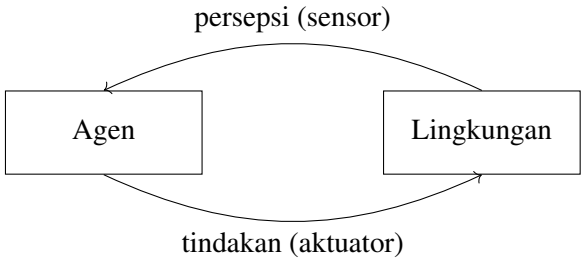
2.12. Sistem Multi-Agen

2.12.1. Definisi Agen dan Sistem Multi-Agen

Definisi 2.12.1 (Wooldridge, 2009) *Agen (agent) adalah entitas komputasi yang mengamati lingkungannya melalui sensor dan bertindak terhadap lingkungan itu melalui aktuator secara otonom — tanpa campur tangan langsung manusia atau agen lain pada setiap langkahnya — untuk mencapai tujuan tertentu.*

Definisi 2.12.2 (Wooldridge, 2009) *Sistem multi-agen (multi-agent system, MAS) adalah sistem yang tersusun atas dua atau lebih agen yang berinteraksi — saling bertukar informasi dan saling memengaruhi tindakan satu sama lain — di dalam lingkungan bersama.*

Gambar 2.6 menggambarkan siklus persepsi–tindakan satu agen terhadap lingkungannya.

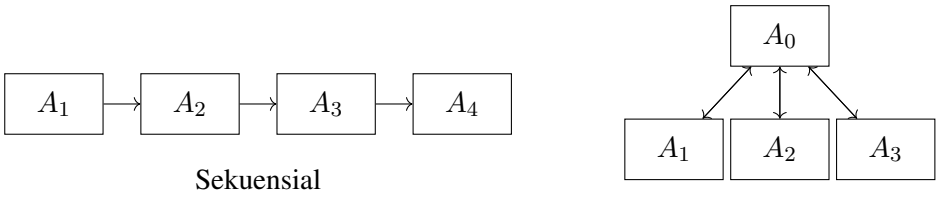


Gambar 2.6 Siklus persepsi–tindakan agen dan lingkungannya [Wooldridge, 2009]. Pada sistem multi-agen, beberapa agen semacam ini berbagi satu lingkungan dan dapat saling memengaruhi.

2.12.2. Topologi Sekuensial dan Hierarkis

Interaksi antaragen pada sistem multi-agen dapat diatur menurut berbagai topologi koordinasi; dua yang paling mendasar adalah sekuensial dan hierarkis [Wooldridge, 2009]. Pada topologi *sekuensial*, agen disusun sebagai rantai: keluaran satu agen menjadi masukan agen berikutnya, sehingga informasi mengalir searah dari agen pertama hingga agen terakhir. Pada topologi *hierarkis*, satu agen berperan sebagai koordinator yang mendelegasikan sub-tugas kepada beberapa agen ba-

wahan dan menggabungkan keluaran mereka. Gambar 2.7 menggambarkan kedua topologi tersebut.



Gambar 2.7 Topologi sekuensial (kiri), rantai agen $A_1 \rightarrow A_2 \rightarrow A_3 \rightarrow A_4$, dan topologi hierarkis (kanan), koordinator A_0 yang mendelegasikan ke A_1, A_2, A_3 dan menggabungkan keluarannya.

BAB III

METODOLOGI PENELITIAN

Penelitian ini membandingkan *Mixture of Inputs* (MoI), yaitu metode *reasoning* melalui representasi laten, dengan tiga metode lain: `raw` (*input-output alignment*), `soft` (*Soft Thinking*), dan `sample` (*discrete token*). Keempatnya diterapkan pada sistem multi-agen yang berkomunikasi melalui transfer *KV-cache*, yang selanjutnya juga dibandingkan dengan sistem multi-agen standar berbasis teks.

3.1. Arsitektur Qwen3-8B

Penelitian ini menggunakan Qwen3-8B, yaitu model bahasa autoregresif dengan arsitektur *Transformer decoder-only* [Yang dkk, 2025]. Model ini memiliki 36 lapisan *decoder* dan dimensi *hidden state* sebesar $d = 4096$. Arsitektur ini berperan dalam menghasilkan representasi internal (*hidden state*) yang digunakan untuk membentuk langkah laten serta menyediakan *key-value cache* (KV-cache) yang memungkinkan informasi dari pemrosesan sebelumnya digunakan kembali sebagai medium komunikasi.

Parameter arsitektur Qwen3-8B yang relevan dengan penelitian ini dirangkum pada Tabel 3.1.

Tabel 3.1 Parameter arsitektur Qwen3-8B yang digunakan dalam penelitian.

Komponen	Nilai	Sumber
Jumlah lapisan L	36	Yang dkk [2025]
Dimensi *hidden state* d	4096	Yang dkk [2025]
Query heads H_q	32	Yang dkk [2025]
Key-value heads H_{kv}	8	Yang dkk [2025]
Dimensi tiap *head* d_k	128	d/H_q
Ukuran kosakata V	151 936	Pengukuran model
Embedding masukan dan keluaran	*Untied*	Yang dkk [2025]
Rata-rata kosinus per baris	0,0041	Pengukuran model
Normalisasi	RMSNorm, *pre-norm*	Yang dkk [2025]

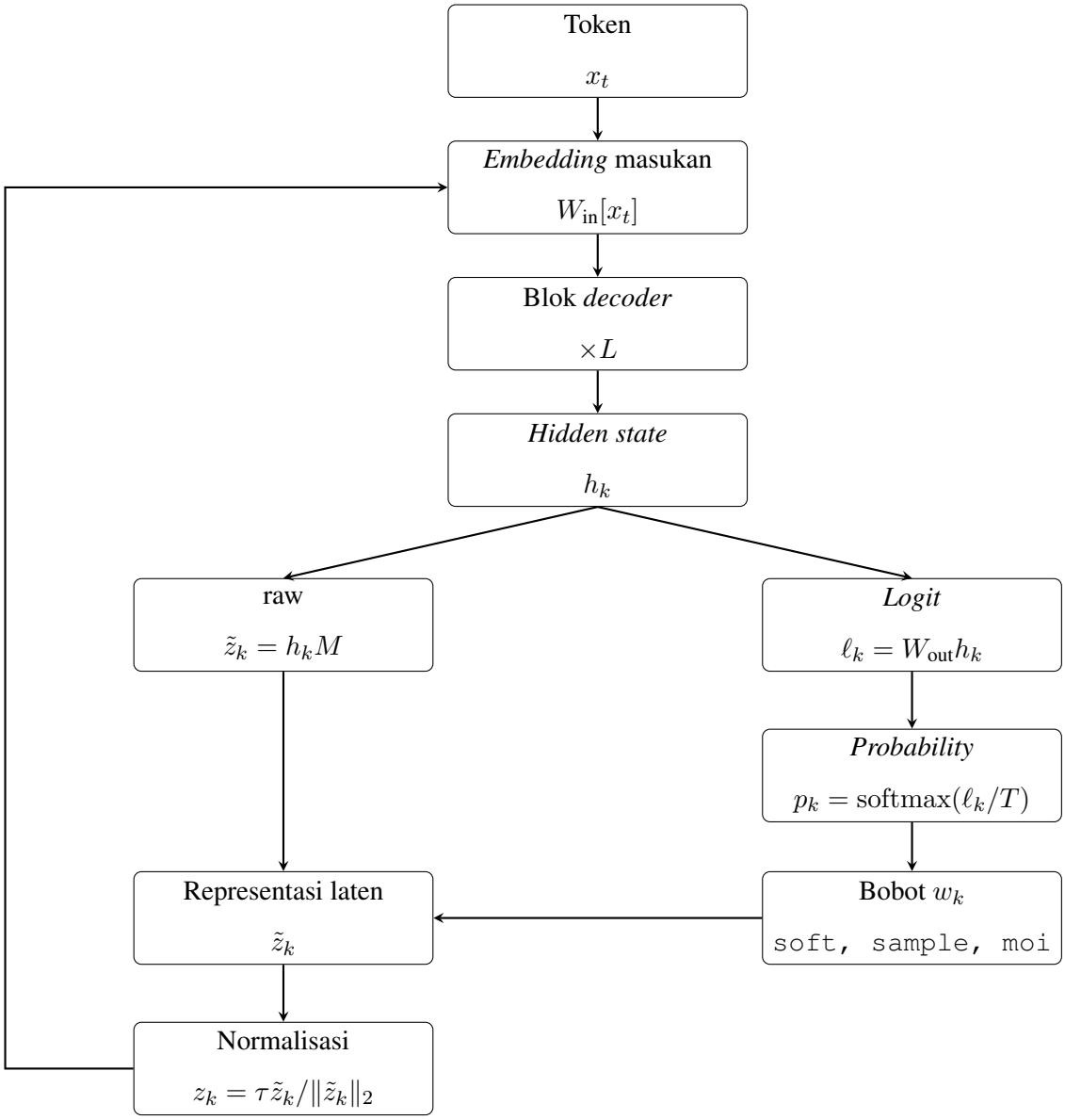
3.1.1. *Hidden State* dan Pembentukan Representasi Laten

Teks masukan terlebih dahulu diubah menjadi urutan token, kemudian setiap token direpresentasikan sebagai vektor melalui matriks *embedding* masukan $W_{\text{in}} \in \mathbb{R}^{V \times d}$. Urutan vektor tersebut diproses melalui seluruh lapisan *decoder* untuk menghasilkan representasi kontekstual. Pada posisi terakhir, representasi setelah lapisan terakhir dinormalisasi menggunakan *Root Mean Square Normalization* (RMSNorm) untuk memperoleh *hidden state* yang digunakan dalam pembentukan langkah laten.

Untuk suatu vektor $x \in \mathbb{R}^d$, RMSNorm didefinisikan sebagai

$$\mathrm{RMSNorm}(x) = \frac{x}{\sqrt{\frac{1}{d}\sum_{j=1}^{d} x_j^2 + \epsilon}} \odot g, \qquad \epsilon = 10^{-6}, \tag{3.1}$$

dengan $g \in \mathbb{R}^d$ merupakan vektor parameter penguatan yang dipelajari, ϵ merupakan konstanta untuk menjaga stabilitas perhitungan, dan $\odot$ menyatakan perkalian Hadamard. Dimensi d menyesuaikan dimensi vektor yang dinormalisasi.



Gambar 3.1 Alur representasi laten pada arsitektur Qwen3.

Jika $x_n^{(L)} \in \mathbb{R}^d$ menyatakan representasi pada posisi terakhir n setelah lapisan ke-L, maka *hidden state* yang digunakan dalam penelitian ini dinyatakan sebagai

$$h_{k-1} = \text{RMSNorm}\left(x_n^{(L)}\right), \qquad h_{k-1} \in \mathbb{R}^d. \tag{3.2}$$

Indeks $k-1$ menunjukkan bahwa representasi tersebut telah tersedia sebelum langkah laten ke-k dibentuk. Metode laten memetakan h_{k-1} menjadi vektor masukan baru z_k, yang kemudian diproses kembali oleh model untuk memperoleh

representasi pada langkah berikutnya.

Matriks W_{in} digunakan untuk mengubah token menjadi vektor embedding. Sebaliknya, matriks proyeksi keluaran $W_{\text{out}} \in \mathbb{R}^{V \times d}$ digunakan untuk mengubah *hidden state* menjadi *logit*. Untuk suatu $h \in \mathbb{R}^d$, diperoleh

$$\ell = W_{\text{out}} h, \tag{3.3}$$

dengan $\ell \in \mathbb{R}^V$ merupakan vektor *logit* untuk seluruh entri kosakata. Distribusi probabilitas token selanjutnya diperoleh dengan menerapkan fungsi *softmax* pada *logit* tersebut. Distribusi ini digunakan oleh metode laten yang membentuk representasi berdasarkan probabilitas token, sedangkan metode `raw` menggunakan W_{in} dan W_{out} untuk menyelaraskan ruang representasi keluaran dengan ruang embedding masukan.

Pada Qwen3-8B, matriks embedding masukan dan proyeksi keluaran tidak menggunakan bobot yang sama (*untied*). Kesamaan arah vektor pada baris yang bersesuaian dapat diperiksa melalui rata-rata kosinus

$$\frac{1}{V} \sum_{i=1}^{V} \cos\left(W_{\text{in}}[i], W_{\text{out}}[i]\right) = 0{,}0041. \tag{3.4}$$

Nilai tersebut menunjukkan bahwa rata-rata kesamaan kosinus antara baris-baris yang bersesuaian relatif dekat dengan nol. Pengukuran ini memberikan gambaran mengenai perbedaan arah kedua ruang representasi.

3.1.2. *Grouped-Query Attention* dan Dimensi *Key-Value*

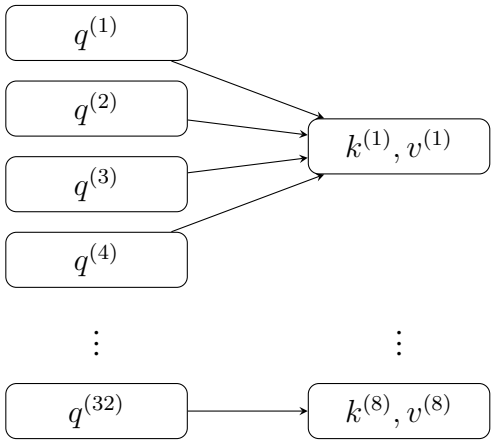
Pada blok *decoder* terdapat mekanisme *Grouped-Query Attention* (GQA) yang berfungsi untuk memproyeksikan representasi masukan menjadi vektor *query*, *key*, dan *value*. Beberapa *query heads* berbagi pasangan *key-value* yang sama sehingga model memiliki $H_q = 32$ *query heads* dan $H_{kv} = 8$ pasangan *key-value heads* [Yang dkk, 2025]. Dimensi setiap *head* ditentukan oleh

$$d_k = \frac{d}{H_q} = \frac{4096}{32} = 128. \tag{3.5}$$

Untuk *query head* ke-η, dengan $\eta \in \{1, \ldots, 32\}$, indeks pasangan *key-value* yang digunakan dapat dinyatakan sebagai

$$g(\eta) = \left\lceil \frac{\eta}{H_q/H_{kv}} \right\rceil = \left\lceil \frac{\eta}{4} \right\rceil, \qquad g(\eta) \in \{1, \ldots, 8\}. \tag{3.6}$$

Persamaan tersebut menunjukkan bahwa setiap empat *query heads* menggunakan pasangan *key-value* yang sama. Ilustrasi sederhana pengelompokan tersebut disajikan pada Gambar 3.2.



Gambar 3.2 Ilustrasi pengelompokan *query heads* pada GQA.

Untuk *query head* ke-η, vektor *query*, *key*, dan *value* diperoleh melalui proyeksi dari vektor masukan yang telah dinormalisasi, yaitu $x \in \mathbb{R}^d$, melalui

$$\begin{aligned} q^{(\eta)} &= W_Q^{(\eta)} x, \\ k^{(g(\eta))} &= W_K^{(g(\eta))} x, \\ v^{(g(\eta))} &= W_V^{(g(\eta))} x, \end{aligned} \tag{3.7}$$

dengan $W_Q^{(\eta)}$, $W_K^{(g(\eta))}$, dan $W_V^{(g(\eta))} \in \mathbb{R}^{d_k \times d}$ merupakan matriks proyeksi yang dipelajari selama pelatihan. Vektor $q^{(\eta)}$, $k^{(g(\eta))}$, dan $v^{(g(\eta))}$ masing-masing berada dalam $\mathbb{R}^{d_k}$.

Dengan $d_k = 128$ dan $H_{kv} = 8$, dimensi gabungan proyeksi *query* adalah

$H_q d_k = 4096$, sedangkan dimensi gabungan proyeksi *key* dan *value* masing-masing adalah $H_{kv} d_k = 1024$. Untuk satu posisi pada satu lapisan, dimensi representasi *key* dan *value* masing-masing dinyatakan sebagai

$$K_t^{(l)}, V_t^{(l)} \in \mathbb{R}^{H_{kv} \times d_k} = \mathbb{R}^{8 \times 128}, \tag{3.8}$$

dengan l menyatakan indeks lapisan.

3.1.3. *Key-Value Cache* (KV-cache)

KV-cache merupakan penyimpanan representasi *key* dan *value* yang dihasilkan pada setiap lapisan selama pemrosesan masukan. Cache ini memungkinkan model menggunakan kembali informasi dari posisi-posisi sebelumnya tanpa harus menghitung ulang representasi *key* dan *value* untuk seluruh posisi tersebut pada setiap langkah.

Untuk lapisan ke-l, setelah t posisi diproses, KV-cache secara konseptual dapat dituliskan sebagai

$$\mathrm{KV}_t^{(l)} = \left\{ \left(k_j^{(l,g)}, v_j^{(l,g)} \right) : 1 \le j \le t,\ 1 \le g \le H_{kv} \right\}. \tag{3.9}$$

Pada persamaan tersebut, j menyatakan indeks posisi dan g menyatakan indeks pasangan *key-value*. Karena Qwen3-8B memiliki $H_{kv} = 8$, setiap posisi baru yang diproses menambahkan delapan pasangan *key-value* ke dalam cache pada masing-masing lapisan. Pertumbuhan tersebut dapat dinyatakan secara konseptual sebagai

$$\mathrm{KV}_{t+1}^{(l)} = \mathrm{KV}_t^{(l)} \parallel \left\{ \left(k_{t+1}^{(l,g)}, v_{t+1}^{(l,g)} \right) : 1 \le g \le 8 \right\}, \tag{3.10}$$

dengan $\parallel$ menyatakan penggabungan sepanjang dimensi posisi.

Dalam penelitian ini, vektor laten juga dimasukkan kembali ke model sebagai masukan pada posisi baru. Pemrosesan tersebut menghasilkan *hidden state* baru sekaligus memperbarui KV-cache. Cache yang terbentuk dapat diteruskan ke-

pada agen berikutnya sebagai konteks internal, sehingga informasi dari pemrosesan sebelumnya dapat digunakan kembali tanpa harus selalu dikonversi menjadi teks. KV-cache tidak menyimpan vektor laten secara langsung, melainkan representasi *key* dan *value* yang dihasilkan oleh lapisan-lapisan model ketika memproses masukan tersebut.

3.2. Komunikasi Antar-Agen

Komunikasi antar-agen digunakan untuk meneruskan informasi dari satu agen ke agen berikutnya dalam sistem multi-agen. Sistem pada penelitian ini terdiri atas empat agen yang bekerja secara berurutan, yaitu *planner*, *critic*, *refiner*, dan *judger*. Rantai agen tersebut dinyatakan sebagai

$$A_1 \longrightarrow A_2 \longrightarrow A_3 \longrightarrow A_4, \tag{3.11}$$

dengan

$$A_1 = \text{Planner}, \quad A_2 = \text{Critic}, \quad A_3 = \text{Refiner}, \quad A_4 = \text{Judger}. \tag{3.12}$$

Setiap agen menerima instruksi berupa *prompt* dan, kecuali agen pertama, informasi yang diteruskan dari agen sebelumnya. Misalkan P_i menyatakan *prompt* untuk agen ke-i, sedangkan $C_i^{(c)}$ menyatakan konteks komunikasi pada medium $c \in \{\texttt{text}, \texttt{kv}\}$. Masukan konseptual agen ke-i dinyatakan sebagai

$$X_i^{(c)} = \left(P_i, C_i^{(c)}\right). \tag{3.13}$$

Keluaran komunikasi agen bergantung pada medium yang digunakan dan dinyatakan sebagai

$$O_i^{(c)} = A_i\left(P_i, C_i^{(c)}\right). \tag{3.14}$$

Pada persamaan (3.14), A_i merupakan notasi konseptual untuk proses pemrosesan oleh agen ke-i, bukan penamaan fungsi implementasi tertentu. Keluaran $O_i^{(c)}$ menyatakan informasi yang dihasilkan untuk diteruskan sesuai dengan medium komunikasi yang digunakan. Untuk agen pertama, tidak terdapat informasi dari agen sebelumnya sehingga konteks awal pada kedua medium adalah

$$C_1^{(c)} = \varnothing. \tag{3.15}$$

Perbedaan kedua medium terletak pada bentuk informasi yang diteruskan. Pada medium `text`, keluaran agen berupa teks yang digunakan sebagai konteks bagi agen berikutnya. Pada medium `kv`, informasi diteruskan melalui *KV-cache* yang terbentuk selama pemrosesan *prompt* dan *latent pass*. Kedua medium menggunakan rantai agen yang sama, tetapi berbeda dalam representasi informasi yang diwariskan.

3.2.1. Komunikasi `text`

Pada medium `text`, komunikasi dilakukan dengan meneruskan keluaran tekstual suatu agen sebagai konteks bagi agen berikutnya. Misalkan S_i menyatakan teks keluaran agen ke-i. Keluaran komunikasi pada medium ini adalah

$$O_i^{(\text{text})} = S_i. \tag{3.16}$$

Agen pertama menghasilkan teks berdasarkan *prompt*-nya tanpa konteks komunikasi dari agen sebelumnya,

$$S_1 = A_1\left(P_1, \varnothing\right). \tag{3.17}$$

Untuk agen ke-i, dengan $i > 1$, konteks komunikasi dibentuk dari keluaran tekstual seluruh agen sebelumnya. Konteks tersebut dibatasi hingga panjang maksimum L dan dinyatakan sebagai

$$C_i^{(\text{text})} = \text{trunc}_L\left(S_1 \| \cdots \| S_{i-1}\right), \qquad i > 1, \tag{3.18}$$

dengan $\|$ menyatakan konkatenasi teks dan trunc_L menyatakan pemangkasan konteks hingga panjang maksimum L. Pada penelitian ini digunakan

$$L = 8000. \tag{3.19}$$

Selanjutnya, keluaran agen ke-i dihasilkan berdasarkan *prompt* dan konteks tekstual yang diterimanya,

$$S_i = A_i\left(P_i, C_i^{(\text{text})}\right), \qquad i > 1. \tag{3.20}$$

Dengan demikian, informasi diteruskan dalam bentuk teks sepanjang rantai agen,

$$S_1 \longrightarrow S_2 \longrightarrow S_3 \longrightarrow S_4. \tag{3.21}$$

Pada medium ini, keluaran teks dari agen-agen sebelumnya menjadi konteks tekstual yang diproses oleh agen berikutnya. Setiap agen memproses konteks tersebut sebagai bagian dari masukan model, tanpa mewarisi *KV-cache* dari agen sebelumnya.

3.2.2. Komunikasi `kv`

Pada medium `kv`, informasi antar-agen diteruskan melalui *KV-cache* yang terbentuk selama pemrosesan *prompt* dan *latent pass*. Cache menyimpan representasi *key* dan *value* pada setiap lapisan model untuk posisi-posisi yang telah diproses. Cache dari agen sebelumnya digunakan sebagai keadaan awal ketika agen berikutnya memproses masukannya. Oleh karena itu, informasi internal yang telah direpresentasikan dalam cache dapat digunakan kembali tanpa perlu menghitung ulang representasi seluruh posisi sebelumnya.

Keluaran komunikasi agen ke-i pada medium ini dinyatakan sebagai

$$O_i^{(\mathrm{kv})} = \mathrm{KV}_i, \tag{3.22}$$

dengan KV_i menyatakan ***KV-cache*** akhir yang dihasilkan oleh pemrosesan agen ke-i. Untuk agen pertama, konteks komunikasi awalnya kosong,

$$C_1^{(\mathrm{kv})} = \varnothing. \tag{3.23}$$

Agen pertama memproses *prompt* dan menjalankan *latent pass* untuk membentuk cache awal,

$$\mathrm{KV}_1 = A_1\left(P_1, \varnothing\right). \tag{3.24}$$

Cache KV_1 menjadi konteks awal bagi agen kedua. Untuk agen ke-i, dengan $i > 1$, cache dari agen sebelumnya diwariskan sebagai konteks komunikasi,

$$C_i^{(\mathrm{kv})} = \mathrm{KV}_{i-1}, \qquad i > 1. \tag{3.25}$$

Persamaan (3.25) menyatakan bahwa agen ke-i menerima cache akhir agen ke-$(i-1)$ sebagai *past KV states*. Agen tersebut kemudian memproses *prompt*-nya dengan menggunakan cache yang diwariskan sebagai keadaan awal. Secara konseptual, keseluruhan pemrosesan agen ke-i dapat dinyatakan sebagai

$$\mathrm{KV}_i = A_i\left(P_i, \mathrm{KV}_{i-1}\right), \qquad i > 1. \tag{3.26}$$

Pada persamaan (3.26), P_i menyatakan *prompt* yang diproses oleh agen saat ini, sedangkan KV_{i-1} menyatakan cache yang diwariskan dari agen sebelumnya. Operator A_i merangkum pemrosesan tersebut, termasuk pemrosesan masukan dan *latent pass* sesuai dengan mode agen yang digunakan, sehingga menghasilkan cache akhir KV_i. Persamaan ini tidak menyatakan penggabungan dua cache yang telah dibentuk secara terpisah. Cache sebelumnya justru digunakan selama model memproses masukan baru.

Struktur internal *KV-cache*. Pada lapisan ke-ℓ, cache yang merepresentasikan t posisi yang telah diproses dinyatakan sebagai

$$\mathrm{KV}_t^{(\ell)} = \left(K_t^{(\ell)}, V_t^{(\ell)}\right), \tag{3.27}$$

dengan $K_t^{(\ell)}$ dan $V_t^{(\ell)}$ masing-masing menyatakan tensor *key* dan *value* untuk posisi-posisi yang telah diproses pada lapisan tersebut. Indeks ℓ menunjukkan lapisan model, sedangkan indeks t menunjukkan panjang urutan posisi yang direpresentasikan dalam cache.

Ketika model memproses satu posisi baru, representasi *key-value* untuk posisi tersebut ditambahkan ke cache. Pembaruan ini dapat dinyatakan sebagai

$$\mathrm{KV}_{t+1}^{(\ell)} = \mathrm{KV}_t^{(\ell)} \| \left(k_{t+1}^{(\ell)}, v_{t+1}^{(\ell)}\right), \tag{3.28}$$

dengan $k_{t+1}^{(\ell)}$ dan $v_{t+1}^{(\ell)}$ menyatakan representasi *key* dan *value* untuk posisi baru. Simbol $\|$ menyatakan konkatenasi sepanjang dimensi posisi, bukan penjumlahan nilai tensor.

Jika pemrosesan agen ke-i menambahkan r_i posisi baru, pertumbuhan panjang cache dapat dituliskan secara konseptual sebagai

$$t_i = t_{i-1} + r_i, \qquad r_i \geq 0. \tag{3.29}$$

dengan r_i menyatakan banyaknya posisi baru yang ditambahkan selama pemrosesan agen ke-i.

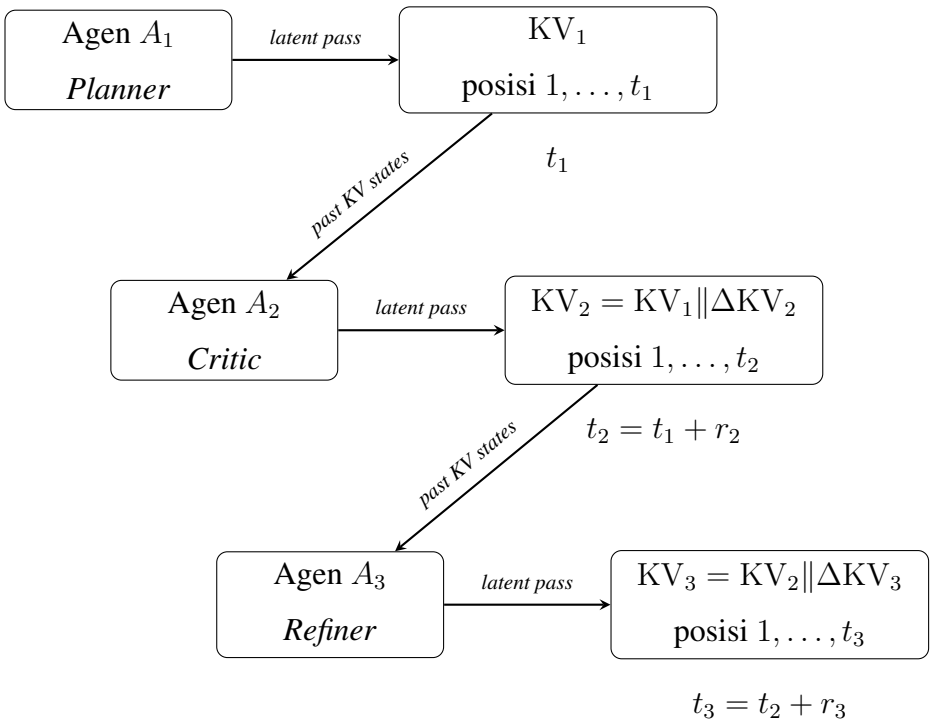
Pertumbuhan cache antar-agen. Pertumbuhan cache di dalam satu agen perlu dibedakan dari pewarisan cache antar-agen. Pada tingkat internal, cache diperpanjang ketika model memproses posisi-posisi baru. Pada tingkat antar-agen, cache akhir dari agen sebelumnya digunakan sebagai keadaan awal untuk pemrosesan agen berikutnya.

Pada lapisan ke-ℓ, hubungan tersebut dapat dinyatakan secara konseptual

sebagai

$$\mathrm{KV}_i^{(\ell)} = \mathrm{KV}_{i-1}^{(\ell)} \| \Delta\mathrm{KV}_i^{(\ell)}, \tag{3.30}$$

dengan $\mathrm{KV}_{i-1}^{(\ell)}$ menyatakan cache yang diwariskan dari agen sebelumnya dan $\Delta\mathrm{KV}_i^{(\ell)}$ menyatakan representasi *key-value* baru yang ditambahkan selama pemrosesan agen ke-i pada lapisan ℓ. Simbol $\|$ menunjukkan penggabungan sepanjang dimensi posisi.



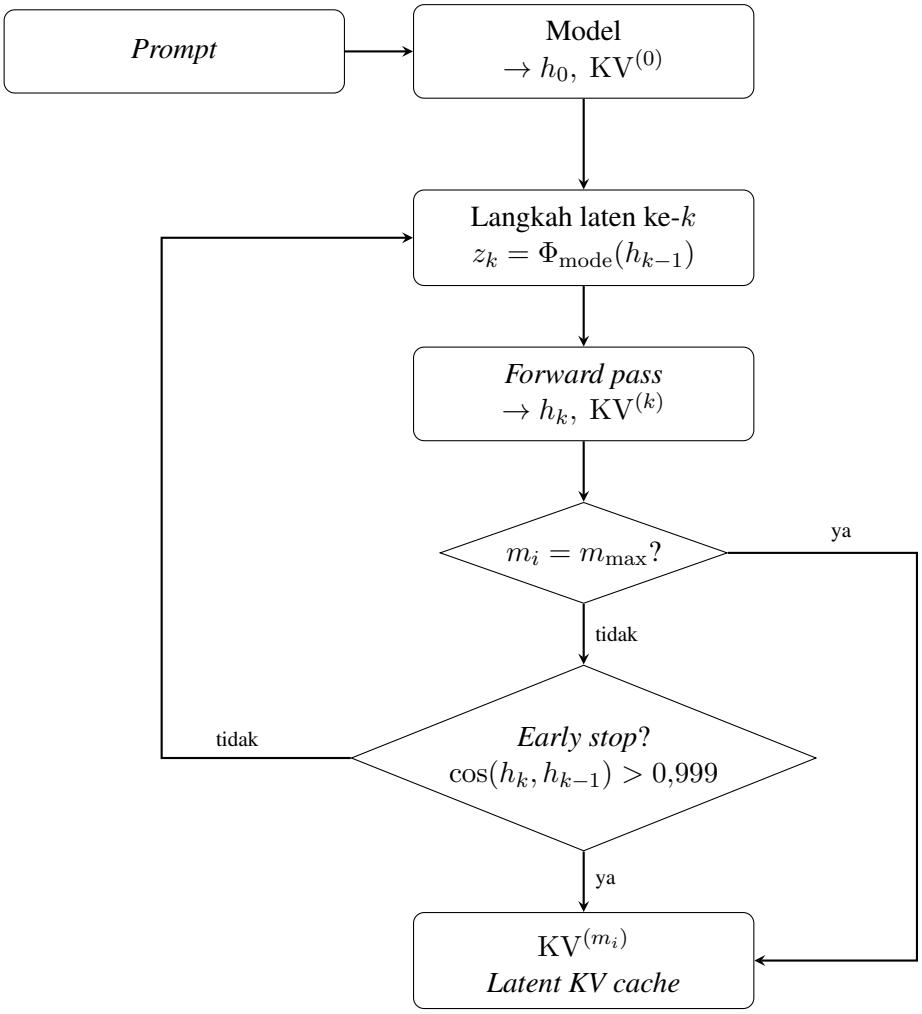
Gambar 3.3 Alur pewarisan dan pertumbuhan *KV-cache* antar-agen. Cache yang diwariskan digunakan sebagai keadaan awal pemrosesan agen berikutnya, sedangkan pemrosesan posisi baru menghasilkan representasi tambahan.

Gambar 3.3 memperlihatkan pewarisan cache secara berurutan. Agen pertama membentuk KV_1. Agen kedua menggunakan cache tersebut sebagai *past KV states* ketika memproses masukannya, lalu menghasilkan KV_2 dan seterusnya.

Pembangkitan teks jawaban dilakukan pada tahap akhir oleh agen *judger*. Oleh karena itu, KV_i pada medium `kv` merepresentasikan keadaan internal hasil pemrosesan agen, bukan keluaran teks yang dihasilkan oleh setiap agen.

3.3. Langkah Laten

Langkah laten memungkinkan model memproses representasi kontinu keluaran (*output*) sebagai masukan (*input*) untuk langkah berikutnya, tanpa harus menghasilkan token diskret pada setiap langkah penalaran. Pada generasi konvensional, model memilih token dari distribusi probabilitas, kemudian menggunakan embedding token tersebut sebagai masukan berikutnya. Sebaliknya, pada penalaran laten, representasi kontinu dapat langsung diberikan kembali kepada model untuk menghasilkan representasi berikutnya. Proses ini diulang hingga memenuhi kriteria penghentian atau mencapai batas maksimum langkah laten.



Gambar 3.4 Alur satu *latent pass* pada satu agen.

3.3.1. Satu Latent Pass

Satu *latent pass* merupakan rangkaian langkah laten yang dijalankan secara berurutan oleh satu agen. Proses dimulai dengan pemrosesan prompt oleh model hingga diperoleh *hidden state* awal h_0 dan KV-cache awal $\mathrm{KV}^{(0)}$. Inisialisasi tersebut dapat dinyatakan sebagai

$$\mathrm{Prompt} \longrightarrow \left(h_0, \mathrm{KV}^{(0)}\right). \tag{3.31}$$

Pada langkah laten ke-k, vektor laten dibentuk dari *hidden state* pada langkah sebelumnya melalui fungsi Φ_{mode}, dengan mode menunjukkan metode pembentukan representasi laten. Selanjutnya, vektor tersebut diproses kembali oleh model menggunakan KV-cache yang telah terbentuk sebelumnya. Kedua proses tersebut dinyatakan sebagai

$$z_k = \Phi_{\mathrm{mode}}(h_{k-1}), \tag{3.32}$$

$$\left(h_k, \mathrm{KV}^{(k)}\right) = f_\theta\left(z_k, \mathrm{KV}^{(k-1)}\right), \tag{3.33}$$

dengan $h_{k-1}, h_k, z_k \in \mathbb{R}^d$ dan f_θ menyatakan satu *forward pass* model. KV-cache pada Persamaan (3.33) digunakan sebagai konteks untuk pemrosesan berikutnya, sehingga informasi dari posisi-posisi sebelumnya tetap tersedia.

Pada setiap langkah, z_k tidak diubah menjadi token diskret, melainkan dimasukkan langsung ke model sebagai representasi kontinu. Pemrosesan ini menghasilkan pasangan key dan value baru pada setiap lapisan model. Secara konseptual, pembaruan KV-cache dapat dituliskan sebagai

$$\mathrm{KV}^{(k)} = \mathrm{KV}^{(k-1)} \parallel \mathrm{KV}(z_k), \tag{3.34}$$

dengan $\parallel$ menyatakan penggabungan sepanjang dimensi urutan dan $\mathrm{KV}(z_k)$ menyatakan pasangan key dan value yang dihasilkan dari pemrosesan z_k. KV-cache

tidak menyimpan vektor laten z_k secara langsung, melainkan representasi key dan value yang dibentuk model dari vektor tersebut.

Setelah proses berhenti pada langkah ke-m_i, satu *latent pass* menghasilkan rangkaian vektor laten

$$Z_i = (z_1, z_2, \ldots, z_{m_i}), \tag{3.35}$$

serta KV-cache akhir

$$\mathrm{KV}_i = \mathrm{KV}^{(m_i)}. \tag{3.36}$$

KV-cache akhir tersebut dapat digunakan sebagai konteks untuk generasi teks pada tahap berikutnya apabila mode komunikasi yang digunakan memerlukan keluaran tekstual. Generasi teks merupakan tahap terpisah dari rangkaian langkah laten itu sendiri.

3.3.2. Penghentian Langkah Laten

Jumlah langkah laten maksimum pada penelitian ini ditetapkan sebesar $m_{\max} = 10$. Namun, satu agen tidak selalu menjalankan seluruh langkah tersebut. Setelah memperoleh *hidden state* baru h_k, model mengevaluasi kemiripannya dengan *hidden state* sebelumnya, h_{k-1}, menggunakan *cosine similarity*:

$$\cos(h_k, h_{k-1}) = \frac{h_k^\top h_{k-1}}{\|h_k\|_2 \|h_{k-1}\|_2}. \tag{3.37}$$

Langkah laten dihentikan apabila

$$\cos(h_k, h_{k-1}) > 0{,}999. \tag{3.38}$$

Kriteria tersebut menunjukkan bahwa dua *hidden state* berturut-turut memiliki arah yang sangat serupa. Jika kriteria tersebut belum terpenuhi, proses dilanjutkan hingga kriteria penghentian terpenuhi atau jumlah langkah mencapai batas

maksimum. Jumlah langkah laten aktual pada agen ke-i dengan demikian memenuhi

$$1 \leq m_i \leq m_{\max}, \qquad m_{\max} = 10. \tag{3.39}$$

3.3.3. Kerangka Metode

Keempat metode pembentukan langkah laten menggunakan *hidden state* yang sama sebagai sumber informasi. Perbedaan antarmetode terletak pada mekanisme pembentukan representasi laten awal $\widetilde{z}_k^{\text{mode}}$. Representasi tersebut kemudian dinormalisasi agar memiliki norma yang sama.

Secara umum, hubungan tersebut dinyatakan sebagai

$$z_k^{\text{mode}} = \tau \frac{\widetilde{z}_k^{\text{mode}}}{\left\| \widetilde{z}_k^{\text{mode}} \right\|_2}, \qquad \text{mode} \in \{\texttt{raw}, \texttt{soft}, \texttt{sample}, \texttt{moi}\}. \tag{3.40}$$

dengan $\tau > 0$ merupakan norma target yang ditetapkan sama untuk seluruh metode. Normalisasi ini menyamakan besar norma representasi yang diberikan kepada model, sehingga perbedaan antarmetode terutama terletak pada arah representasi yang dihasilkan. Norma yang digunakan adalah norma Euclidean (L_2), yaitu $\|\widetilde{z}_k^{\text{mode}}\|_2 = \sqrt{\sum_{j=1}^{d} (\widetilde{z}_{k,j}^{\text{mode}})^2}$, yang mengukur panjang vektor representasi laten sebelum normalisasi.

Untuk metode `soft`, `sample`, dan `moi`, representasi laten awal dibentuk melalui matriks embedding masukan $W_{\text{in}} \in \mathbb{R}^{V \times d}$. Jika $w_k \in \mathbb{R}^V$ menyatakan bobot atas kosakata, maka

$$\widetilde{z}_k^{\text{mode}} = W_{\text{in}}^{\top} w_k, \qquad \text{mode} \in \{\texttt{soft}, \texttt{sample}, \texttt{moi}\}. \tag{3.41}$$

Dengan demikian, ketiga metode tersebut membentuk representasi laten sebagai kombinasi berbobot dari embedding token. Perbedaannya terletak pada cara memperoleh w_k, yang dijelaskan pada bagian masing-masing metode.

Sementara itu, metode `raw` membentuk representasi laten awal dengan memetakan *hidden state* menggunakan matriks penyelarasan $W_a \in \mathbb{R}^{d \times d}$.

3.4. Metode Pembentukan Langkah Laten

Penelitian ini membandingkan empat metode pembentukan langkah laten, yaitu `raw`, `soft`, `sample`, dan `moi`. Keempat metode tersebut mengubah *hidden state* yang dihasilkan model menjadi representasi kontinu yang dapat diberikan kembali kepada model. Perbedaan utama antarmetode terletak pada cara pembentukan representasi tersebut. Metode `raw` menggunakan pemetaan langsung dari ruang keluaran model menuju ruang embedding masukan, sedangkan metode `soft`, `sample`, dan `moi` memanfaatkan distribusi probabilitas token untuk membentuk representasi laten.

Misalkan $h_{k-1} \in \mathbb{R}^d$ merupakan *hidden state* sebelum pembentukan langkah laten ke-k, dengan d menyatakan dimensi tersembunyi model. Matriks embedding masukan dinyatakan sebagai $W_{\text{in}} \in \mathbb{R}^{V \times d}$, sedangkan matriks proyeksi keluaran dinyatakan sebagai $W_{\text{out}} \in \mathbb{R}^{V \times d}$, dengan V merupakan ukuran kosakata.

Untuk metode yang menggunakan distribusi token, *logit* dan distribusi probabilitas token dihitung melalui

$$\boldsymbol{\ell}_k = W_{\text{out}} h_{k-1}, \tag{3.42}$$

$$p_k = \text{softmax}\left(\frac{\boldsymbol{\ell}_k}{T}\right), \tag{3.43}$$

dengan $T > 0$ merupakan temperatur yang mengatur bentuk distribusi probabilitas. Vektor $p_k \in \mathbb{R}^V$ memenuhi $p_{k,i} \geq 0$ dan $\sum_{i=1}^{V} p_{k,i} = 1$. Distribusi tersebut menjadi dasar pembentukan representasi pada metode `soft`, `sample`, dan `moi`.

Representasi sebelum normalisasi dibentuk menggunakan mekanisme yang berbeda pada setiap metode. Metode `raw` memetakan *hidden state* melalui matriks penyelarasan, sedangkan metode `soft`, `sample`, dan `moi` membentuk kombinasi berbobot dari embedding token. Setelah representasi awal diperoleh, representasi

akhir setiap metode dihitung menggunakan normalisasi pada Persamaan (3.40).

3.4.1. Metode `raw`

Zou dkk. [Zou dkk, 2025] memperkenalkan mekanisme penyelarasan (*alignment*) input-output untuk mengatasi ketidaksesuaian antara ruang representasi *hidden state* dan ruang *embedding* masukan. *Hidden state* dari lapisan terakhir tidak selalu sesuai dengan distribusi representasi yang diharapkan ketika dimasukkan kembali ke lapisan awal model. Oleh karena itu, LatentMAS menggunakan matriks penyelarasan $W_a \in \mathbb{R}^{d\times d}$ untuk memetakan *hidden state* menuju ruang embedding masukan.

Matriks penyelarasan ditentukan dengan meminimalkan selisih antara matriks embedding masukan W_{in} dan hasil transformasi matriks keluaran W_{out}. Fungsi objektif yang diusulkan dalam paper LatentMAS adalah

$$\min_{W_a} \|W_{\text{out}}W_a - W_{\text{in}}\|_F^2 \,. \tag{3.44}$$

Norma Frobenius kuadrat pada Persamaan (3.44) menjumlahkan kuadrat selisih seluruh elemen kedua matriks. Dengan demikian, optimasi mencari W_a yang menghasilkan proyeksi $W_{\text{out}}W_a$ sedekat mungkin dengan W_{in}.

Untuk memperoleh solusi, fungsi objektif tersebut dapat diperluas menjadi

$$\begin{aligned} J(W_a) &= \|W_{\text{out}}W_a - W_{\text{in}}\|_F^2 && (3.45)\\ &= \operatorname{tr}\left[(W_{\text{out}}W_a - W_{\text{in}})^\top (W_{\text{out}}W_a - W_{\text{in}})\right] && (3.46)\\ &= \operatorname{tr}\left(W_a^\top W_{\text{out}}^\top W_{\text{out}} W_a\right) - 2\operatorname{tr}\left(W_a^\top W_{\text{out}}^\top W_{\text{in}}\right) + \operatorname{tr}\left(W_{\text{in}}^\top W_{\text{in}}\right). && (3.47) \end{aligned}$$

Turunkan fungsi objektif terhadap W_a dan menyamakan gradiennya dengan nol, diperoleh

$$\frac{\partial J}{\partial W_a} = 2W_{\text{out}}^{\top} W_{\text{out}} W_a - 2W_{\text{out}}^{\top} W_{\text{in}} = 0. \tag{3.48}$$

Persamaan tersebut menghasilkan persamaan normal

$$W_{\text{out}}^{\top} W_{\text{out}} W_a = W_{\text{out}}^{\top} W_{\text{in}}. \tag{3.49}$$

Apabila $W_{\text{out}}^{\top} W_{\text{out}}$ invertibel, solusi minimumnya adalah

$$W_a = \left(W_{\text{out}}^{\top} W_{\text{out}}\right)^{-1} W_{\text{out}}^{\top} W_{\text{in}}. \tag{3.50}$$

Dalam penelitian ini, solusi diterapkan menggunakan regularisasi ridge untuk meningkatkan stabilitas numerik. Fungsi objektif yang digunakan menjadi

$$J_\lambda(W_a) = \|W_{\text{out}} W_a - W_{\text{in}}\|_F^2 + \lambda \|W_a\|_F^2, \qquad \lambda > 0. \tag{3.51}$$

Suku regularisasi mencegah solusi bergantung secara berlebihan pada matriks yang sulit diinvers. Dengan menurunkan $J_\lambda(W_a)$ terhadap W_a dan menyamakan gradiennya dengan nol, diperoleh

$$\frac{\partial J_\lambda}{\partial W_a} = 2W_{\text{out}}^{\top} W_{\text{out}} W_a - 2W_{\text{out}}^{\top} W_{\text{in}} + 2\lambda W_a = 0, \tag{3.52}$$

sehingga

$$\left(W_{\text{out}}^{\top} W_{\text{out}} + \lambda I\right) W_a = W_{\text{out}}^{\top} W_{\text{in}}. \tag{3.53}$$

Matriks penyelarasan yang digunakan dalam penelitian ini adalah

$$\boxed{W_a = \left(W_{\text{out}}^{\top} W_{\text{out}} + \lambda I\right)^{-1} W_{\text{out}}^{\top} W_{\text{in}}}, \qquad \lambda = 10^{-5}. \tag{3.54}$$

Representasi sebelum normalisasi dibentuk dengan memetakan *hidden state* dari langkah sebelumnya, yaitu

$$\widetilde{z}_k^{\text{raw}} = W_a^\top h_{k-1}. \tag{3.55}$$

Representasi akhir diperoleh dengan menerapkan Persamaan (3.40). Berbeda dari metode `soft`, `sample`, dan `moi`, metode `raw` memetakan *hidden state* secara langsung menggunakan matriks penyelarasan, tanpa membentuk kombinasi embedding berdasarkan distribusi probabilitas token.

3.4.2. Metode `soft`

Metode `soft` menggunakan distribusi probabilitas token secara langsung sebagai bobot untuk menggabungkan embedding masukan. Distribusi p_k dihitung menggunakan Persamaan (3.43). Representasi sebelum normalisasi kemudian diperoleh melalui

$$\widetilde{z}_k^{\text{soft}} = W_{\text{in}}^\top p_k = \sum_{i=1}^{V} p_{k,i} W_{\text{in}}[i], \tag{3.56}$$

dengan $W_{\text{in}}[i] \in \mathbb{R}^d$ menyatakan embedding masukan untuk token ke-i. Karena setiap komponen p_k tidak negatif dan jumlah seluruh komponennya sama dengan satu, representasi tersebut merupakan kombinasi konveks dari embedding token dalam kosakata.

Representasi akhir metode `soft`, yaitu z_k^{soft}, diperoleh dengan menerapkan Persamaan (3.40) pada $\widetilde{z}_k^{\text{soft}}$. Metode ini mempertahankan kontribusi seluruh distribusi probabilitas token, alih-alih hanya menggunakan embedding dari satu token terpilih.

“‘latex

3.4.3. Metode `sample`

Metode `sample` memilih satu token secara acak berdasarkan distribusi probabilitas yang dihasilkan model. Misalkan ℓ_k merupakan vektor logit pada langkah laten ke-k dan $T > 0$ merupakan parameter suhu. Probabilitas setiap token dihitung menggunakan fungsi softmax:

$$p_k = \mathrm{softmax}\left(\frac{\ell_k}{T}\right), \tag{3.57}$$

dengan $p_k \in \Delta^{V-1}$ merupakan distribusi probabilitas atas V kemungkinan token. Token terpilih dinyatakan dengan indeks y_k dan diperoleh melalui sampling dari distribusi kategorik,

$$y_k \sim \mathrm{Cat}(p_k). \tag{3.58}$$

Artinya, token dengan probabilitas lebih tinggi memiliki peluang lebih besar untuk terpilih, tetapi token dengan probabilitas lebih rendah tetap dapat terpilih.

Indeks token terpilih kemudian direpresentasikan sebagai vektor satu-*hot* $e_{y_k} \in \mathbb{R}^V$. Komponen ke-i dari vektor tersebut didefinisikan sebagai

$$e_{k,i} = \mathbb{1}(i = y_k), \qquad i = 1, \ldots, V, \tag{3.59}$$

dengan $\mathbb{1}(\cdot)$ merupakan fungsi indikator. Hanya komponen yang sesuai dengan token terpilih bernilai satu, sedangkan komponen lainnya bernilai nol.

Misalkan $W_{\mathrm{in}} \in \mathbb{R}^{V \times d}$ merupakan matriks embedding masukan, dengan setiap barisnya menyatakan embedding satu token. Representasi laten sebelum normalisasi diperoleh dengan memilih embedding token yang terpilih:

$$\widetilde{z}_k^{\mathrm{sample}} = W_{\mathrm{in}}^{\top} e_{y_k} = W_{\mathrm{in}}[y_k]. \tag{3.60}$$

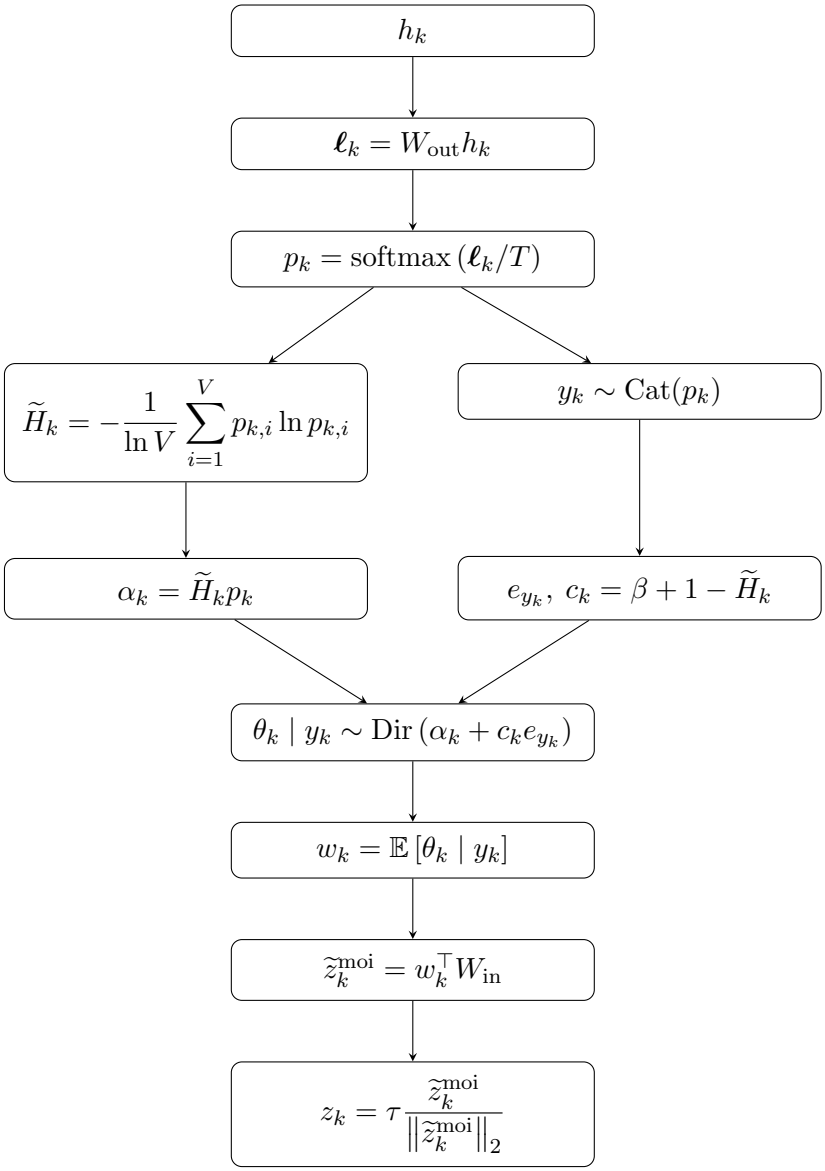
Persamaan tersebut menunjukkan bahwa perkalian matriks dengan vektor satu-*hot* memilih satu baris matriks embedding. Dalam implementasi, operasi ini dilakukan secara langsung melalui pengindeksan matriks menggunakan indeks token hasil

sampling, tanpa membentuk vektor satu-*hot* secara eksplisit.

Representasi akhir z_k^{sample} diperoleh dengan normalisasi sebagaimana pada Persamaan (3.40). Dengan demikian, metode `sample` menggunakan satu embedding token diskret pada setiap langkah laten, berbeda dari metode `soft` yang menggabungkan embedding seluruh token berdasarkan bobot probabilitasnya. “‘

3.4.4. Metode `moi`

Metode *Mixture of Inputs* (MoI) menggabungkan informasi dari distribusi probabilitas token dan token hasil *sampling* untuk membentuk representasi masukan kontinu. Berbeda dari metode `soft` yang menggunakan distribusi probabilitas secara langsung dan metode `sample` yang menggunakan embedding satu token terpilih, MoI membentuk bobot campuran melalui estimasi rerata *posterior* Bayesian. Mekanisme ini memanfaatkan distribusi Dirichlet untuk menggabungkan distribusi prediktif model dengan informasi dari token hasil *sampling* [Zhuang dkk, 2025].



Gambar 3.5 Alur pembentukan langkah laten menggunakan *Mixture of Inputs*.

Distribusi probabilitas token pada langkah ke-k, yaitu p_k, dihitung menggunakan Persamaan (3.43). Untuk mengukur tingkat ketidakpastian distribusi tersebut, terlebih dahulu didefinisikan entropi Shannon sebagai

$$H(p_k) = -\sum_{i=1}^{V} p_{k,i} \ln p_{k,i}, \tag{3.61}$$

dengan V menyatakan jumlah token dalam kosakata. Entropi mengukur ketidakpastian distribusi probabilitas, dimana nilainya rendah ketika probabilitas ter-

konsentrasi pada beberapa token dan tinggi ketika probabilitas tersebar lebih merata. Nilai entropi maksimum tercapai ketika seluruh token memiliki probabilitas yang sama, yaitu $H_{\max} = \ln V$. Oleh karena itu, entropi dinormalisasi terhadap nilai maksimumnya agar berada pada skala yang sama, terlepas dari jumlah kategori dalam distribusi.

$$\widetilde{H}_k = \frac{H(p_k)}{\ln V} = -\frac{1}{\ln V} \sum_{i=1}^{V} p_{k,i} \ln p_{k,i}, \qquad 0 \leq \widetilde{H}_k \leq 1. \tag{3.62}$$

Batas bawah $\widetilde{H}_k = 0$ terjadi ketika seluruh probabilitas terkonsentrasi pada satu token, sedangkan batas atas $\widetilde{H}_k = 1$ terjadi ketika distribusi seragam. Dengan demikian, $\widetilde{H}_k$ digunakan sebagai ukuran tingkat ketidakpastian distribusi prediktif model.

Selanjutnya, MoI membentuk *prior* Dirichlet untuk memodelkan vektor bobot campuran $\theta_k \in \Delta^{V-1}$, dengan Δ^{V-1} menyatakan simpleks probabilitas berdimensi $V - 1$. Parameter konsentrasi *prior* ditentukan berdasarkan entropi ternormalisasi dan distribusi prediktif [Zhuang dkk, 2025]:

$$\theta_k \sim \mathrm{Dir}(\boldsymbol{\alpha}_k), \qquad \boldsymbol{\alpha}_k = \widetilde{H}_k p_k. \tag{3.63}$$

Komponen parameter konsentrasi tersebut memenuhi $\alpha_{k,i} = \widetilde{H}_k p_{k,i}$. Jumlah seluruh parameter konsentrasi adalah

$$\alpha_{k,0} = \sum_{i=1}^{V} \alpha_{k,i} \tag{3.64}$$

$$= \sum_{i=1}^{V} \widetilde{H}_k p_{k,i} \tag{3.65}$$

$$= \widetilde{H}_k \sum_{i=1}^{V} p_{k,i} \tag{3.66}$$

$$= \widetilde{H}_k, \tag{3.67}$$

karena $\sum_{i=1}^{V} p_{k,i} = 1$. Untuk distribusi Dirichlet dengan parameter konsentrasi $\boldsymbol{\alpha}_k$, rerata setiap komponennya diberikan oleh

$$\mathbb{E}[\theta_{k,i}] = \frac{\alpha_{k,i}}{\alpha_{k,0}}. \tag{3.68}$$

Dengan demikian, untuk $\widetilde{H}_k > 0$, diperoleh

$$\mathbb{E}[\theta_{k,i}] = \frac{\widetilde{H}_k p_{k,i}}{\widetilde{H}_k} = p_{k,i}. \tag{3.69}$$

Hasil ini menunjukkan bahwa p_k merupakan rerata atau pusat dari *prior* Dirichlet, sedangkan $\widetilde{H}_k$ mengatur total konsentrasi atau kekuatan informasi *prior*. Ketika entropi meningkat, total konsentrasi *prior* juga meningkat. Sebaliknya, ketika entropi menurun menuju nol, total konsentrasi *prior* ikut menurun. Token diskret y_k disampling dari distribusi probabilitas

$$y_k \sim \mathrm{Cat}(p_k), \tag{3.70}$$

dengan $\mathrm{Cat}(p_k)$ menyatakan distribusi kategorik yang memiliki V kemungkinan token. Token hasil *sampling* direpresentasikan menggunakan vektor indikator atau *one-hot vector*,

$$e_{y_k} = (e_{k,1}, e_{k,2}, \ldots, e_{k,V}), \tag{3.71}$$

dengan setiap komponennya didefinisikan sebagai

$$e_{k,i} = \mathbb{1}(i = y_k). \tag{3.72}$$

Fungsi indikator $\mathbb{1}(i = y_k)$ bernilai satu apabila indeks token ke-i sama dengan token hasil *sampling* dan bernilai nol selainnya. Oleh karena itu, hanya satu komponen e_{y_k} yang bernilai satu, sehingga

$$\sum_{i=1}^{V} e_{k,i} = 1. \tag{3.73}$$

Informasi token hasil *sampling* kemudian dimasukkan ke dalam parameter konsentrasi melalui *pseudo-count*. Dalam formulasi MoI, vektor tambahan tersebut ditentukan sebagai

$$c_k = \left(\beta + 1 - \widetilde{H}_k\right) e_{y_k}, \tag{3.74}$$

dengan $\beta \geq 0$ merupakan parameter yang mengatur kontribusi tambahan bagi token hasil *sampling*. Jumlah seluruh komponen *pseudo-count* dapat diturunkan menggunakan Persamaan (3.73):

$$\sum_{i=1}^{V} c_{k,i} = \left(\beta + 1 - \widetilde{H}_k\right) \sum_{i=1}^{V} e_{k,i} \tag{3.75}$$

$$= \beta + 1 - \widetilde{H}_k. \tag{3.76}$$

Persamaan tersebut menunjukkan bahwa kontribusi total *pseudo-count* berbanding terbalik dengan entropi. Ketika entropi rendah, kontribusi token hasil *sampling* menjadi lebih besar. Sebaliknya, ketika entropi tinggi, kontribusi tambahan tersebut menjadi lebih kecil. Dengan demikian, mekanisme ini menyesuaikan keseimbangan antara distribusi prediktif dan token terpilih berdasarkan tingkat ketidakpastian model.

Karena distribusi Dirichlet bersifat konjugat terhadap distribusi kategorik, penambahan informasi observasi dalam bentuk *pseudo-count* mempertahankan bentuk distribusi *posterior* sebagai distribusi Dirichlet. Dengan parameterisasi di atas, diperoleh

$$\theta_k \mid y_k \sim \mathrm{Dir}(\boldsymbol{\alpha}_k + c_k). \tag{3.77}$$

Memasukkan Persamaan (3.63) dan Persamaan (3.74) menghasilkan para-

meter *posterior*

$$\boldsymbol{\alpha}_k + c_k = \widetilde{H}_k p_k + \left(\beta + 1 - \widetilde{H}_k\right) e_{y_k}. \tag{3.78}$$

Distribusi *posterior* dapat dituliskan secara eksplisit sebagai

$$\boxed{\theta_k \mid y_k \sim \mathrm{Dir}\left(\widetilde{H}_k p_k + \left(\beta + 1 - \widetilde{H}_k\right) e_{y_k}\right)}. \tag{3.79}$$

MoI menggunakan rerata *posterior*. untuk distribusi Dirichlet dengan vektor parameter konsentrasi $\boldsymbol{\gamma} = (\gamma_1, \ldots, \gamma_V)$, rerata komponennya adalah

$$\mathbb{E}[\theta_i] = \frac{\gamma_i}{\sum_{j=1}^{V} \gamma_j}. \tag{3.80}$$

Dengan demikian, bobot campuran setiap token ditentukan oleh

$$w_{k,i} = \mathbb{E}[\theta_{k,i} \mid y_k] = \frac{\alpha_{k,i} + c_{k,i}}{\sum_{j=1}^{V} (\alpha_{k,j} + c_{k,j})}. \tag{3.81}$$

Penyebut pada Persamaan (3.81) merupakan jumlah seluruh parameter konsentrasi *posterior*. Berdasarkan Persamaan (3.67) dan Persamaan (3.76), nilainya adalah

$$\sum_{j=1}^{V} (\alpha_{k,j} + c_{k,j}) = \sum_{j=1}^{V} \alpha_{k,j} + \sum_{j=1}^{V} c_{k,j} \tag{3.82}$$

$$= \widetilde{H}_k + \left(\beta + 1 - \widetilde{H}_k\right) \tag{3.83}$$

$$= \beta + 1. \tag{3.84}$$

Masukkan parameter *prior*, *pseudo-count*, dan penyebut tersebut ke dalam Persamaan (3.81), diperoleh

$$w_{k,i} = \frac{\widetilde{H}_k p_{k,i} + \left(\beta + 1 - \widetilde{H}_k\right) e_{k,i}}{\beta + 1}. \tag{3.85}$$

Dalam bentuk vektor, persamaan utama pembentukan bobot MoI dapat ditulis sebagai [Zhuang dkk, 2025]

$$\boxed{w_k = \mathbb{E}[\theta_k \mid y_k] = \frac{\widetilde{H}_k p_k + \left(\beta + 1 - \widetilde{H}_k\right) e_{y_k}}{\beta + 1}}. \tag{3.86}$$

Persamaan (3.86) menunjukkan bahwa bobot MoI menggabungkan dua sumber informasi, yaitu distribusi probabilitas penuh p_k dan vektor *one-hot* e_{y_k}. Besarnya kontribusi masing-masing sumber ditentukan oleh entropi ternormalisasi dan parameter β. Karena kedua sumber tersebut memiliki komponen nonnegatif dan jumlah komponen masing-masing sama dengan satu, w_k juga memiliki komponen nonnegatif dengan jumlah satu. Dengan demikian, w_k merupakan vektor bobot yang dapat digunakan untuk membentuk kombinasi konveks dengan embedding token.

Dalam penelitian ini, parameter MoI ditetapkan sebesar $\beta = 1$. Dengan memasukkan nilai tersebut ke dalam Persamaan (3.86), bobot campuran menjadi

$$w_k = \frac{\widetilde{H}_k p_k + \left(2 - \widetilde{H}_k\right) e_{y_k}}{2}. \tag{3.87}$$

Representasi sebelum normalisasi kemudian dibentuk melalui kombinasi bobot w_k dan embedding masukan adalah

$$\widetilde{z}_k^{\mathrm{moi}} = W_{\mathrm{in}}^{\top} w_k = \sum_{i=1}^{V} w_{k,i} W_{\mathrm{in}}[i]. \tag{3.88}$$

Representasi akhir z_k^{moi} diperoleh dengan menerapkan Persamaan (3.40) pada $\widetilde{z}_k^{\mathrm{moi}}$. Melalui formulasi tersebut, MoI mempertahankan informasi distribusional dari model sekaligus memberikan kontribusi kepada token hasil *sampling*. Meskipun pembentukannya melibatkan token diskret, representasi yang diberikan kembali kepada model tetap berupa vektor kontinu.

3.5. Rancangan Eksperimen dan Analisis

3.5.1. Rancangan Eksperimen

Penelitian ini menggunakan eksperimen terkendali untuk membandingkan formulasi representasi langkah laten dan medium komunikasi antar-agen dalam *Multi-Agent System* (MAS). Rancangan eksperimen dibentuk dari dua aspek, yaitu medium komunikasi dan formulasi pembentukan representasi langkah laten.

- **Medium komunikasi:** berbasis teks dan berbasis *KV-cache*.
- **Formulasi langkah laten (pada medium *KV-cache*):** `raw`, `soft`, `sample`, dan `moi`.

Kondisi eksperimen terdiri atas satu kondisi komunikasi berbasis teks dan empat kondisi berbasis *KV-cache*:

- Pada kondisi `text`, informasi antar-agen diteruskan sebagai keluaran tekstual sehingga tidak ada persamaan Φ terpisah.
- Pada keempat kondisi lainnya, informasi diteruskan melalui *KV-cache* dan representasi langkah laten dibentuk menggunakan persamaan Φ yang sesuai.

Struktur ini memungkinkan pengaruh medium komunikasi diuji dengan membandingkan kondisi `text` terhadap kelompok kondisi *KV-cache*, sedangkan perbedaan antarformulasi laten dianalisis dengan membandingkan keempat kondisi *KV-cache*.

Secara ringkas, struktur perlakuan dapat dinyatakan sebagai

$$\mathcal{C} = \{\texttt{text}, \texttt{raw}, \texttt{soft}, \texttt{sample}, \texttt{moi}\}, \tag{3.89}$$

dengan

$$\begin{aligned}
\texttt{text}: &\quad \text{medium teks},\\
\texttt{raw}: &\quad \textit{KV-cache} \text{ dengan } \Phi_{\texttt{raw}},\\
\texttt{soft}: &\quad \textit{KV-cache} \text{ dengan } \Phi_{\texttt{soft}},\\
\texttt{sample}: &\quad \textit{KV-cache} \text{ dengan } \Phi_{\texttt{sample}},\\
\texttt{moi}: &\quad \textit{KV-cache} \text{ dengan } \Phi_{\texttt{moi}}.
\end{aligned} \tag{3.90}$$

Seluruh kondisi dijalankan menggunakan *backbone* Qwen3-8B dan rantai agen yang sama, yaitu *planner* $\rightarrow$ *critic* $\rightarrow$ *refiner* $\rightarrow$ *judger*. Untuk kondisi yang menggunakan langkah laten, setiap agen menjalankan paling banyak $m = 10$ langkah laten.

Faktor-faktor yang tidak menjadi objek perlakuan dijaga tetap sepanjang eksperimen utama. Faktor tersebut mencakup model yang digunakan, struktur rantai agen, jumlah maksimum langkah laten, parameter suhu pembentukan distribusi langkah laten, parameter β pada *Mixture of Inputs*, *temperature* pembuatan keluaran teks, batas panjang keluaran, kriteria penghentian dini. Penetapan tersebut dilakukan agar perbedaan hasil antarkondisi dapat dikaitkan dengan perlakuan yang dibandingkan, bukan dengan perubahan konfigurasi eksperimen.

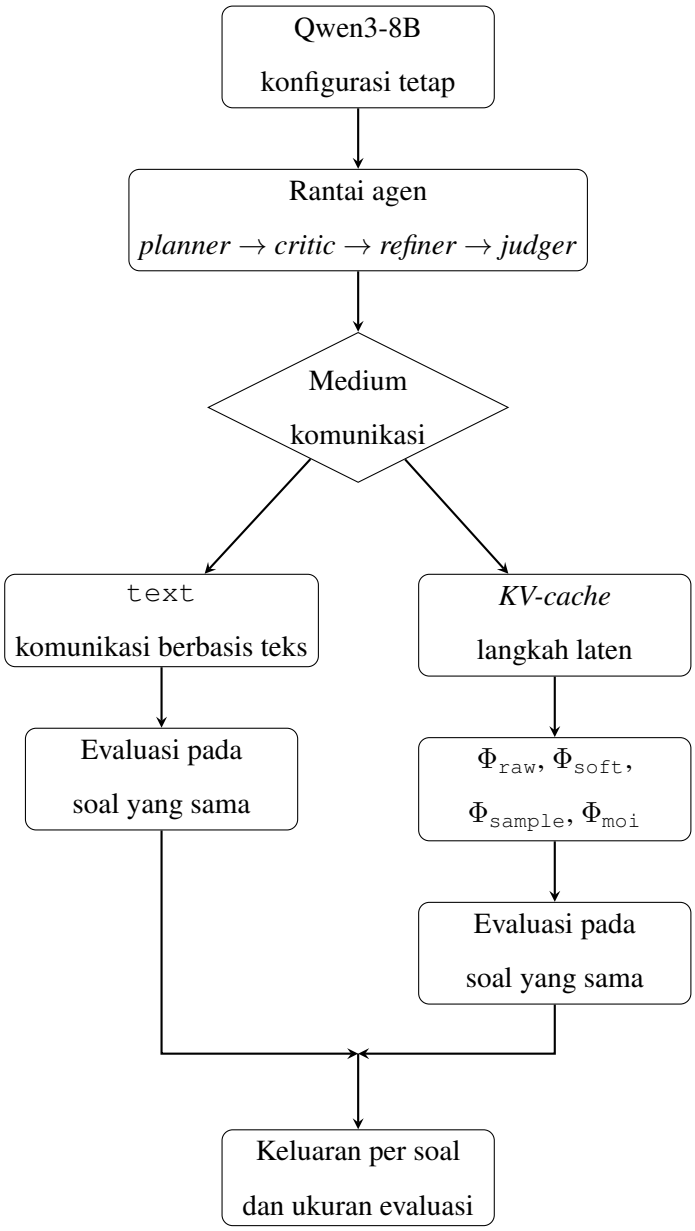
Evaluasi dilakukan pada GSM8K, ARC-Challenge, dan HumanEval+. Ketiga *benchmark* digunakan secara konsisten pada seluruh kondisi dalam *benchmark* yang sama. Pada setiap *benchmark*, himpunan soal yang digunakan juga dibuat identik untuk seluruh kondisi. Dengan demikian, apabila kondisi $c_1, c_2 \in \mathcal{C}$ dibandingkan pada soal ke-k, hasil keduanya membentuk satu pasangan observasi,

$$\left(Y_{k,c_1}, Y_{k,c_2}\right), \qquad k = 1, \ldots, n, \tag{3.91}$$

dengan $Y_{k,c}$ menyatakan hasil evaluasi kondisi c pada soal ke-k. Struktur berpasangan tersebut digunakan karena setiap kondisi dievaluasi pada objek soal yang sama, sehingga perbedaan tingkat kesulitan antar himpunan soal tidak menjadi sumber perbedaan antarkondisi.

Secara keseluruhan, rancangan eksperimen dapat diringkas dalam alur beri-

kut.



Gambar 3.6 Rancangan eksperimen penelitian.

3.5.2. Pengukuran dan Analisis Kinerja

Kinerja setiap kondisi eksperimen dievaluasi berdasarkan akurasi, *format rate*, jumlah token keluaran, dan waktu inferensi. Keempat ukuran dihitung pada tingkat soal sehingga hasil dari soal yang sama pada kondisi berbeda dapat dibandingkan secara berpasangan. Akurasi digunakan untuk mengukur proporsi jawaban

benar, sedangkan *format rate* digunakan untuk mengukur proporsi keluaran yang memenuhi format evaluasi. Definisi kedua ukuran tersebut mengacu pada metrik yang telah dibahas pada Bab 3.5.3..

Untuk kondisi c pada suatu *benchmark*, dengan N soal, akurasi dihitung sebagai

$$A_c = \frac{1}{N} \sum_{i=1}^{N} I(Y_{ic} = 1), \tag{3.92}$$

dengan $Y_{ic} = 1$ apabila jawaban pada soal ke-i dinyatakan benar dan $Y_{ic} = 0$ apabila dinyatakan salah. Sementara itu, *format rate* dihitung sebagai

$$F_c = \frac{1}{N} \sum_{i=1}^{N} I(R_{ic} = 1), \tag{3.93}$$

dengan $R_{ic} = 1$ apabila keluaran dapat diproses oleh prosedur evaluasi dan $R_{ic} = 0$ sebaliknya.

Jumlah token keluaran dan waktu inferensi digunakan sebagai ukuran efisiensi. Untuk masing-masing kondisi, nilai yang dilaporkan adalah rata-rata per soal,

$$\bar{T}_c = \frac{1}{N} \sum_{i=1}^{N} T_{ic}, \qquad \bar{W}_c = \frac{1}{N} \sum_{i=1}^{N} W_{ic}, \tag{3.94}$$

dengan T_{ic} menyatakan jumlah token keluaran dan W_{ic} menyatakan waktu inferensi untuk soal ke-i pada kondisi c.

Perbandingan berpasangan. Karena himpunan soal yang digunakan sama pada seluruh kondisi dalam satu *benchmark*, perbandingan dilakukan berdasarkan pasangan hasil pada soal yang sama. Untuk dua kondisi c_1 dan c_2, pasangan pengamatan ke-i dituliskan sebagai

$$\left(Y_{ic_1}, Y_{ic_2}\right), \qquad i = 1, \ldots, N. \tag{3.95}$$

Pendekatan berpasangan digunakan agar perbedaan tingkat kesulitan antar-

soal tidak diperlakukan sebagai variasi antarperlakuan.

Uji McNemar eksak. Perbedaan akurasi antara dua kondisi dianalisis menggunakan uji McNemar eksak. Untuk setiap pasangan kondisi, banyaknya soal diklasifikasikan menjadi empat kategori a, b, c, d, dengan b dan c menunjukkan hasil yang berbeda antara kedua kondisi. Di bawah hipotesis nol

$$H_0 : \pi_b = \pi_c, \tag{3.96}$$

jumlah pasangan diskordan $b + c$ digunakan sebagai ukuran sampel pada distribusi binomial dengan probabilitas $1/2$. Nilai-p diperoleh secara eksak dari distribusi tersebut. Pendekatan ini digunakan untuk menghindari ketergantungan pada aproksimasi χ^2 ketika jumlah pasangan diskordan relatif kecil.

***Bootstrap* berpasangan.** Untuk jumlah token dan waktu inferensi, ketidakpastian perbedaan antarkondisi diukur menggunakan *bootstrap* berpasangan. Untuk setiap soal didefinisikan selisih

$$D_i = X_{ic_1} - X_{ic_2}, \qquad i = 1, \ldots, N, \tag{3.97}$$

dengan X berupa jumlah token atau waktu inferensi. Indeks soal kemudian di-*resample* dengan pengembalian sehingga pasangan pengamatan tetap dipertahankan. Statistik selisih dihitung pada setiap sampel ulang dan interval kepercayaan 95% diperoleh dari persentil ke-$2{,}5$ dan ke-$97{,}5$ distribusi *bootstrap*. Prosedur ini mengikuti konsep *bootstrap* yang telah dijelaskan pada Bab **??**.

Koreksi perbandingan berganda. Pengujian dilakukan untuk beberapa pasangan kondisi dan *benchmark*. Oleh karena itu, nilai-p dari pengujian yang termasuk dalam satu kelompok perbandingan disesuaikan menggunakan koreksi perbandingan berganda sebelum interpretasi signifikansi dilakukan. Hasil analisis dilaporkan bersama estimasi perbedaan dan interval kepercayaannya sehingga keputusan tidak

hanya didasarkan pada nilai-p.

3.5.3. Benchmark dan Metrik Evaluasi

Kemampuan model bahasa pada tugas bernalar diukur lewat tolok ukur (*benchmark*) baku: kumpulan soal beserta cara penilaian jawabannya yang telah distandardisasi agar hasil dapat dibandingkan antarpenelitian. Berikut tiga tolok ukur yang merentang jenis keluaran yang dituntut, dari bilangan tunggal hingga kode program yang harus dieksekusi.

GSM8K [Cobbe dkk, 2021] berisi soal cerita matematika tingkat sekolah dasar–menengah yang memerlukan beberapa langkah penalaran aritmetika berurutan; jawaban akhir berupa satu bilangan.

ARC-Challenge [Clark dkk, 2018] (*AI2 Reasoning Challenge*) berisi soal sains pilihan ganda yang dipilih karena tidak terjawab benar oleh pencocokan kata kunci sederhana, sehingga menuntut penalaran; jawaban berupa satu simbol pilihan.

HumanEval-Plus [Chen dkk, 2021, Liu dkk, 2023] berisi soal pemrograman berupa tanda tangan fungsi beserta deskripsi tugasnya; jawaban berupa kode program yang harus lolos serangkaian kasus uji unit. Akhiran "Plus" menyatakan perluasan kasus uji oleh Liu dkk [2023] yang menutup celah himpunan uji asli [Chen dkk, 2021], sehingga solusi yang tampak benar namun keliru pada kasus tepi lebih mungkin terdeteksi.

Ketiga tolok ukur di atas dinilai dengan metrik berikut.

Definisi 3.5.1 *Untuk N soal dengan hasil benar/salah, akurasi adalah proporsi jawaban benar,*

$$\textit{akurasi} = \frac{1}{N}\left|\{\, i : \textit{jawaban ke-}i\ \textit{benar}\,\}\right|. \tag{3.98}$$

Definisi 3.5.2 (Chen dkk, 2021) *Untuk satu soal pemrograman, dibangkitkan $n \geq k$ sampel kode, dan $c \leq n$ di antaranya lolos seluruh kasus uji. Pass@k adalah peluang paling sedikit satu dari k sampel yang dipilih acak (tanpa pengembalian)*

dari n sampel tersebut lolos, diestimasi tak bias lewat

$$pass@k = \mathop{\mathbb{E}}_{soal}\left[1 - \frac{\binom{n-c}{k}}{\binom{n}{k}}\right]. \tag{3.99}$$

Definisi 3.5.3 *Format rate adalah proporsi keluaran model yang mengikuti format yang diminta (misalnya dapat diurai menjadi jawaban akhir atau kode yang sintaksisnya sahih), terlepas dari benar-salahnya isi jawaban. Metrik ini memisahkan kegagalan mengikuti format keluaran dari kegagalan substansi jawaban.*

Definisi 3.5.4 *Biaya komputasi suatu metode diukur lewat banyaknya token yang diproses (masukan dan keluaran) dan/atau waktu inferensi (wall-clock time) per soal — proksi baku untuk membandingkan efisiensi antarmetode, karena tidak bergantung pada perangkat keras tertentu (peluang token) atau langsung mencerminkan waktu tunggu pengguna (waktu inferensi).*

3.5.4. Pengukuran Geometris

Pengukuran geometris digunakan untuk mengetahui posisi representasi langkah laten yang dihasilkan oleh setiap formulasi relatif terhadap ruang representasi token yang tersedia bagi model.

Objek Representasi

Keempat formulasi langkah laten menggunakan *hidden state* yang sama sebagai sumber informasi. Untuk suatu *hidden state* h, persamaan mode menghasilkan representasi laten

$$z^{\mathrm{mode}} = \Phi_{\mathrm{mode}}(h), \qquad \mathrm{mode} \in \{\mathrm{raw}, \mathrm{soft}, \mathrm{sample}, \mathrm{moi}\}. \tag{3.100}$$

Dalam eksperimen, representasi tersebut telah dinormalisasi menggunakan norma target τ, sehingga

$$z^{\mathrm{mode}} = \tau \frac{\widetilde{z}^{\mathrm{mode}}}{\left\|\widetilde{z}^{\mathrm{mode}}\right\|_2}, \qquad \left\|z^{\mathrm{mode}}\right\|_2 = \tau. \tag{3.101}$$

Normalisasi ini yang memungkinkan perbandingan antarmetode difokuskan pada arah representasi. Dengan norma yang sama, perbedaan panjang vektor tidak menjadi sumber perbedaan pada ukuran geometris yang digunakan.

Ruang Pembanding

Representasi laten dibandingkan dengan ruang *input embedding* model. Misalkan ukuran kosakata adalah V dan dimensi representasi adalah d. Matriks *input embedding* dituliskan sebagai

$$W_{\mathrm{in}} = \begin{bmatrix} W_{\mathrm{in}}[1]^{\mathsf{T}} \\ W_{\mathrm{in}}[2]^{\mathsf{T}} \\ \vdots \\ W_{\mathrm{in}}[V]^{\mathsf{T}} \end{bmatrix} \in \mathbb{R}^{V \times d}, \tag{3.102}$$

dengan

$$W_{\mathrm{in}}[i] \in \mathbb{R}^d \tag{3.103}$$

merupakan vektor *embedding* untuk token ke-i.

Ruang pembanding tersebut dipilih karena z^{mode} digunakan kembali sebagai representasi masukan pada langkah laten berikutnya. Dengan demikian, pertanyaan geometris yang diuji adalah seberapa dekat representasi laten tersebut dengan representasi token yang memang dapat diterima oleh model.

Hal ini berbeda dari membandingkan dengan W_{out}. Matriks W_{out} digunakan untuk memetakan *hidden state* menjadi *logit*, sedangkan W_{in} merupakan ruang tempat vektor representasi token diberikan kepada model. Karena W_{in} dan W_{out} tidak diikat pada model yang digunakan, keduanya tidak diasumsikan sebagai ruang geometris yang sama.

Kemiripan Kosinus

Kedekatan antara dua vektor diukur menggunakan kemiripan kosinus. Untuk dua vektor tidak nol $u, v \in \mathbb{R}^d$, kemiripan kosinus didefinisikan sebagai

$$\cos(u, v) = \frac{u^\mathsf{T} v}{\|u\|_2 \|v\|_2}. \tag{3.104}$$

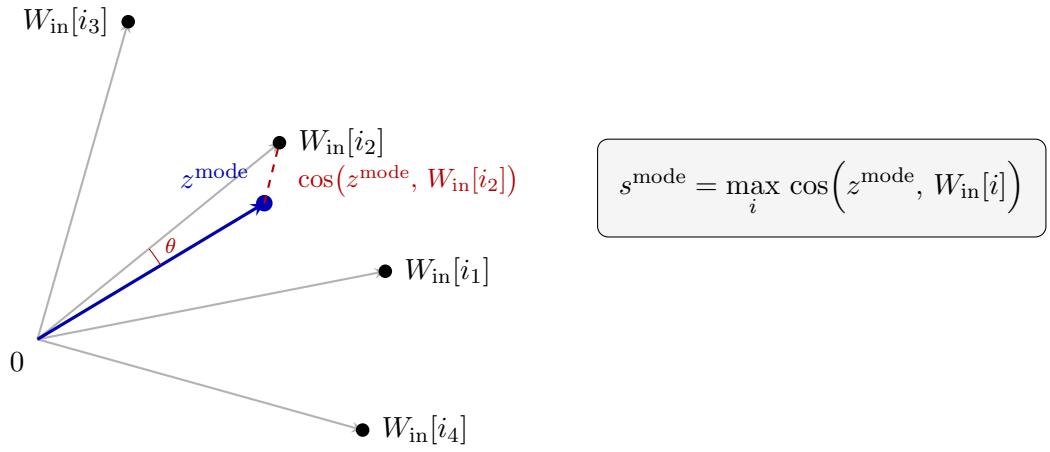
Persamaan (3.104) membandingkan arah kedua vektor. Secara khusus, apabila

$$u^\mathsf{T} v = \|u\|_2 \|v\|_2, \tag{3.105}$$

maka

$$\cos(u, v) = 1, \tag{3.106}$$

yang menunjukkan bahwa kedua vektor memiliki arah yang sama. Sebaliknya, nilai yang semakin kecil menunjukkan bahwa arah kedua vektor semakin berbeda.



Gambar 3.7 Ilustrasi konseptual pengukuran geometris. Vektor z^{mode} dibandingkan dengan seluruh vektor *input embedding* $W_{\text{in}}[i]$, kemudian kemiripan kosinus terbesar digunakan sebagai ukuran kedekatan geometris.

Untuk sebuah representasi laten z^{mode}, kemiripan terhadap seluruh kosakata

dihitung dengan

$$s_i^{\text{mode}} = \cos\left(z^{\text{mode}}, W_{\text{in}}[i]\right), \qquad i = 1, \ldots, V. \tag{3.107}$$

Karena tujuan pengukuran adalah mengetahui apakah terdapat setidaknya satu *embedding* token yang memiliki arah dekat dengan representasi laten, nilai maksimum digunakan sebagai ukuran kedekatan geometris,

$$s^{\text{mode}} = \max_{1 \le i \le V} \cos\left(z^{\text{mode}}, W_{\text{in}}[i]\right). \tag{3.108}$$

Persamaan (3.108) tidak menyatakan bahwa token dengan kemiripan maksimum tersebut merupakan token yang benar atau token yang kemudian dihasilkan model. Ukuran tersebut hanya menyatakan apakah terdapat arah pada ruang *embedding* token yang dekat dengan representasi laten.

Nilai s^{mode} yang semakin mendekati 1 menunjukkan bahwa representasi laten semakin dekat secara arah dengan setidaknya satu *embedding* token. Sebaliknya, nilai yang lebih rendah menunjukkan bahwa representasi tersebut lebih jauh dari seluruh arah *embedding* token.

Agregasi Pengukuran

Satu soal menghasilkan beberapa representasi laten karena setiap agen dapat menjalankan beberapa langkah laten. Oleh karena itu, nilai s^{mode} pada Persamaan (3.108) terlebih dahulu dihitung pada setiap langkah laten.

Misalkan q menyatakan indeks soal, a indeks agen, dan k indeks langkah laten. Kedekatan geometris pada satu posisi representasi didefinisikan sebagai

$$s_{qak}^{\text{mode}} = \max_{1 \le i \le V} \cos\left(z_{qak}^{\text{mode}}, W_{\text{in}}[i]\right). \tag{3.109}$$

Untuk memperoleh satu ukuran geometris pada tingkat agen dan soal, nilai tersebut dirata-ratakan sepanjang langkah laten yang benar-benar dihasilkan. Jika K_{qa}^{mode} adalah jumlah langkah laten pada agen a untuk soal q, maka

$$\bar{s}_{qa}^{\text{mode}} = \frac{1}{K_{qa}^{\text{mode}}} \sum_{k=1}^{K_{qa}^{\text{mode}}} s_{qak}^{\text{mode}}. \tag{3.110}$$

Agregasi selanjutnya dilakukan pada tingkat agen. Dengan A menyatakan jumlah agen dalam rantai,

$$\bar{s}_{q}^{\text{mode}} = \frac{1}{A} \sum_{a=1}^{A} \bar{s}_{qa}^{\text{mode}}. \tag{3.111}$$

Nilai $\bar{s}_{q}^{\text{mode}}$ kemudian menjadi ukuran geometris pada tingkat soal. Untuk N soal, ukuran agregat suatu persamaan dihitung sebagai

$$\overline{s}^{\text{mode}} = \frac{1}{N} \sum_{q=1}^{N} \bar{s}_{q}^{\text{mode}}. \tag{3.112}$$

Dengan struktur tersebut, agregasi dilakukan secara bertahap,

$$s_{qak}^{\text{mode}} \longrightarrow \bar{s}_{qa}^{\text{mode}} \longrightarrow \bar{s}_{q}^{\text{mode}} \longrightarrow \overline{s}^{\text{mode}}. \tag{3.113}$$

Urutan tersebut mempertahankan unit soal sebagai unit observasi pada analisis selanjutnya. Hal ini memungkinkan ukuran geometris dibandingkan dengan hasil keluaran pada soal yang sama.

Hubungan dengan Keluaran

Ukuran geometris digunakan sebagai variabel representasi, sedangkan keluaran tugas digunakan sebagai ukuran kinerja. Untuk soal ke-q, keberhasilan keluaran dituliskan sebagai variabel indikator

$$Y_{q}^{\text{mode}} = \mathbb{1}\left[\text{keluaran ke-}q\text{ benar}\right]. \tag{3.114}$$

Dengan demikian, setiap soal memiliki pasangan

$$\left(\bar{s}_{q}^{\text{mode}}, Y_{q}^{\text{mode}}\right), \qquad q = 1, \ldots, N. \tag{3.115}$$

Pasangan tersebut digunakan untuk melihat apakah perubahan karakteristik geometris muncul bersamaan dengan perubahan pada keluaran. Pada tingkat agregat, akurasi tetap dihitung dari

$$A^{\mathrm{mode}} = \frac{1}{N}\sum_{q=1}^{N} Y_q^{\mathrm{mode}}, \tag{3.116}$$

sedangkan karakteristik geometris diringkas melalui

$$\overline{s}^{\mathrm{mode}} = \frac{1}{N}\sum_{q=1}^{N} \bar{s}_q^{\mathrm{mode}}. \tag{3.117}$$

Perbandingan A^{mode} dan $\overline{s}^{\mathrm{mode}}$ digunakan sebagai analisis deskriptif untuk melihat apakah formulasi dengan karakteristik geometris yang berbeda juga menunjukkan pola kinerja yang berbeda. Hubungan tersebut tidak dimaksudkan sebagai hubungan sebab-akibat. Kedekatan geometris hanya memberikan bukti mengenai karakteristik representasi yang dapat digunakan untuk membantu menjelaskan perbedaan keluaran.

3.5.5. Interpolasi Geometris

Pengukuran geometris pada Subbab 3.5.4. memberikan ukuran kedekatan masing-masing representasi laten terhadap ruang *embedding* masukan. Namun, perbandingan antara dua persamaan hanya memberikan informasi pada dua titik representasi. Untuk mengamati perubahan kinerja sepanjang perpindahan dari formulasi *raw* menuju formulasi *soft*, digunakan interpolasi geometris.

Interpolasi dilakukan pada keluaran akhir kedua formulasi, yaitu $\Phi_{\texttt{raw}}(h)$ dan $\Phi_{\texttt{soft}}(h)$. Kedua vektor tersebut telah dinormalisasi menggunakan norma yang sama τ sebagaimana pada Pers. (3.101). Untuk $\alpha \in [0, 1]$, mula-mula dibentuk vektor campuran

$$u_\alpha = (1-\alpha)\Phi_{\texttt{raw}}(h) + \alpha\Phi_{\texttt{soft}}(h). \tag{3.118}$$

Vektor campuran tersebut kemudian dinormalisasi kembali ke norma τ,

$$\boxed{\Phi^{(\alpha)}_{\text{mix}}(h) = \tau \frac{u_\alpha}{\|u_\alpha\|}} \qquad u_\alpha \neq \mathbf{0}. \tag{3.119}$$

Pers. (3.118) menentukan arah pencampuran antara kedua representasi, sedangkan Pers. (3.119) memastikan bahwa representasi yang digunakan pada setiap nilai α memiliki magnitudo yang sama. Dengan demikian,

$$\left\| \Phi^{(\alpha)}_{\text{mix}}(h) \right\| = \tau, \qquad \forall \alpha \in [0, 1]. \tag{3.120}$$

Pilihan ini membuat perubahan representasi sepanjang interpolasi dapat dipandang terutama sebagai perubahan arah pada ruang laten.

Titik ujung interpolasi. Konstruksi pada Pers. (3.119) mempertahankan kedua formulasi asal secara tepat pada titik ujung $\alpha = 0$ dan $\alpha = 1$. Untuk $\alpha = 0$, diperoleh

$$\begin{aligned} u_0 &= (1 - 0)\Phi_{\texttt{raw}}(h) + 0\Phi_{\texttt{soft}}(h) \\ &= \Phi_{\texttt{raw}}(h). \end{aligned} \tag{3.121}$$

Karena $\|\Phi_{\texttt{raw}}(h)\| = \tau$, maka

$$\begin{aligned} \Phi^{(0)}_{\text{mix}}(h) &= \tau \frac{\Phi_{\texttt{raw}}(h)}{\|\Phi_{\texttt{raw}}(h)\|} \\ &= \tau \frac{\Phi_{\texttt{raw}}(h)}{\tau} \\ &= \Phi_{\texttt{raw}}(h). \end{aligned} \tag{3.122}$$

Dengan cara yang sama, untuk $\alpha = 1$,

$$u_1 = \Phi_{\texttt{soft}}(h), \tag{3.123}$$

$$\begin{aligned}\Phi^{(1)}_{\text{mix}}(h) &= \tau \frac{\Phi_{\texttt{soft}}(h)}{\|\Phi_{\texttt{soft}}(h)\|} \\ &= \Phi_{\texttt{soft}}(h).\end{aligned} \tag{3.124}$$

Dengan demikian,

$$\boxed{\Phi^{(0)}_{\text{mix}}(h) = \Phi_{\texttt{raw}}(h), \qquad \Phi^{(1)}_{\text{mix}}(h) = \Phi_{\texttt{soft}}(h)}. \tag{3.125}$$

Pers. (3.125) menunjukkan bahwa interpolasi tidak menghasilkan perubahan pada kedua kondisi pembatas. Kondisi $\alpha = 0$ identik dengan formulasi *raw*, sedangkan $\alpha = 1$ identik dengan formulasi *soft*. Oleh karena itu, kedua titik ujung dapat menggunakan hasil eksperimen yang telah diperoleh sebelumnya.

Lintasan geometris. Selanjutnya, misalkan

$$a = \Phi_{\texttt{raw}}(h), \qquad b = \Phi_{\texttt{soft}}(h), \qquad \|a\| = \|b\| = \tau. \tag{3.126}$$

Sebelum normalisasi, u_α merupakan kombinasi linear dari a dan b. Oleh karena itu,

$$u_\alpha = (1-\alpha)a + \alpha b \in \operatorname{span}\{a, b\}. \tag{3.127}$$

Ketika u_α dinormalisasi ke norma τ, setiap titik pada ruas garis yang menghubungkan a dan b diproyeksikan kembali ke bola berjari-jari τ. Karena seluruh ruas tersebut berada pada bidang yang dibentuk oleh a dan b, lintasan hasil normalisasi berada pada irisan bidang tersebut dengan bola, yaitu suatu lingkaran besar (*great circle*).

Dengan demikian, untuk $u_\alpha \neq \mathbf{0}$,

$$\Phi_{\text{mix}}^{(\alpha)}(h) \in \left\{ z \in \mathbb{R}^d : \|z\| = \tau \right\}, \tag{3.128}$$

dan lintasan $\alpha \mapsto \Phi_{\text{mix}}^{(\alpha)}(h)$ bergerak dari $\Phi_{\texttt{raw}}(h)$ menuju $\Phi_{\texttt{soft}}(h)$ pada bola tersebut.

Kasus $u_\alpha = \mathbf{0}$ hanya dapat terjadi ketika kedua vektor berlawanan arah. Karena $\|a\| = \|b\| = \tau$, kondisi tersebut mensyaratkan

$$b = -a, \qquad \alpha = \frac{1}{2}, \tag{3.129}$$

yang ekuivalen dengan

$$\cos(a, b) = -1. \tag{3.130}$$

Pada kondisi tersebut normalisasi pada Pers. (3.119) tidak terdefinisi. Oleh karena itu, interpolasi hanya didefinisikan untuk nilai α yang menghasilkan $u_\alpha \neq \mathbf{0}$.

Hubungan α dengan perpindahan sudut. Parameter α menentukan bobot pencampuran, tetapi tidak secara langsung menentukan jarak geometris pada bola. Untuk melihat hubungan tersebut, misalkan θ adalah sudut antara a dan b,

$$\theta = \arccos\left(\frac{a^\top b}{\|a\|\|b\|} \right). \tag{3.131}$$

Definisikan vektor satuan

$$\hat{a} = \frac{a}{\tau}, \tag{3.132}$$

serta $\hat{n}$ sebagai vektor satuan yang berada pada $\text{span}\{a, b\}$ dan tegak lurus terhadap $\hat{a}$. Dengan demikian, b dapat dituliskan sebagai

$$b = \tau \left(\cos\theta\, \hat{a} + \sin\theta\, \hat{n} \right). \tag{3.133}$$

Substitusi Pers. (3.133) ke Pers. (3.118) menghasilkan

$$\begin{aligned} u_\alpha &= (1-\alpha)\tau\hat{a} + \alpha\tau\left(\cos\theta\,\hat{a} + \sin\theta\,\hat{n}\right) \\ &= \tau\left[\left((1-\alpha) + \alpha\cos\theta\right)\hat{a} + \alpha\sin\theta\,\hat{n}\right]. \end{aligned} \tag{3.134}$$

Jika $\varphi(\alpha)$ menyatakan sudut antara a dan $\Phi^{(\alpha)}_{\text{mix}}(h)$, maka dari komponen yang sejajar dan tegak lurus terhadap $\hat{a}$ diperoleh

$$\boxed{\tan\varphi(\alpha) = \frac{\alpha\sin\theta}{(1-\alpha) + \alpha\cos\theta}}. \tag{3.135}$$

Pers. (3.135) menunjukkan bahwa perubahan α tidak secara umum menghasilkan perubahan sudut yang linear. Sebagai contoh, pada $\alpha = \frac{1}{2}$,

$$\begin{aligned} \tan\varphi\left(\frac{1}{2}\right) &= \frac{\frac{1}{2}\sin\theta}{\frac{1}{2} + \frac{1}{2}\cos\theta} \\ &= \frac{\sin\theta}{1+\cos\theta} \\ &= \tan\frac{\theta}{2}. \end{aligned} \tag{3.136}$$

Sehingga

$$\varphi\left(\frac{1}{2}\right) = \frac{\theta}{2}. \tag{3.137}$$

Hasil tersebut menunjukkan bahwa titik tengah pencampuran setelah normalisasi berada pada titik tengah busur antara kedua representasi. Namun, di luar titik tengah tersebut, jarak sudut tidak berubah secara linear terhadap α. Oleh karena itu, α diperlakukan sebagai parameter pencampuran, bukan sebagai ukuran jarak geometris.

Kisi interpolasi. Eksperimen interpolasi menggunakan kisi

$$\boxed{\alpha \in \{0,\ 0{,}25,\ 0{,}50,\ 0{,}75,\ 1\}.} \tag{3.138}$$

Nilai $\alpha = 0$ dan $\alpha = 1$ masing-masing merepresentasikan formulasi *raw* dan *soft* berdasarkan Pers. (3.125). Dengan demikian, hanya tiga nilai tengah yang memerlukan kondisi eksperimen tambahan.

Kisi tersebut memberikan tiga titik di antara kedua formulasi, yaitu $\alpha = 0{,}25$, $\alpha = 0{,}50$, dan $\alpha = 0{,}75$. Ketiga titik tersebut digunakan untuk mengamati bentuk perubahan kinerja sepanjang lintasan, bukan hanya perbedaan pada kedua titik ujung.

Interpolasi dijalankan pada HumanEval+ dengan menggunakan pertanyaan yang sama seperti eksperimen utama. Pemilihan ini mengikuti rancangan eksperimen yang menggunakan interpolasi sebagai analisis tambahan pada kondisi dengan perbedaan kinerja antara *raw* dan formulasi berbasis token yang lebih jelas.

Pengukuran posisi geometris sepanjang interpolasi. Karena α merupakan bobot pencampuran dan bukan ukuran jarak geometris, posisi representasi sepanjang lintasan diukur kembali menggunakan ukuran geometris pada Subbab 3.5.4.. Untuk setiap α, didefinisikan

$$s(\alpha) = \max_{1 \le i \le V} \cos\left(\Phi_{\text{mix}}^{(\alpha)}(h), W_{\text{in}}[i]\right). \tag{3.139}$$

Dengan demikian, terdapat dua besaran yang berbeda. Parameter α menentukan bagaimana representasi *raw* dan *soft* dicampurkan, sedangkan $s(\alpha)$ mengukur posisi representasi hasil pencampuran relatif terhadap ruang *embedding* token.

Hubungan keduanya kemudian dapat digunakan untuk membandingkan perubahan keluaran model dengan perubahan posisi geometris. Secara khusus, untuk setiap nilai α dicatat keluaran model dan ukuran geometris $s(\alpha)$, sehingga pasangan pengamatan dapat dituliskan sebagai

$$\left(s(\alpha), Y^{(\alpha)}\right), \tag{3.140}$$

dengan $Y^{(\alpha)}$ menyatakan indikator keberhasilan keluaran pada nilai α tersebut. Analisis ini digunakan untuk mengamati apakah perubahan kinerja berjalan searah dengan perubahan kedekatan representasi terhadap ruang *embedding* token.

DAFTAR PUSTAKA

Agresti, A. (2013). *Categorical Data Analysis* (3rd ed.). Hoboken: John Wiley & Sons.

Ainslie, J., Lee-Thorp, J., de Jong, M., Zemlyanskiy, Y., Lebrón, F., & Sanghai, S. (2023). GQA: Training generalized multi-query transformer models from multi-head checkpoints. *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 4895–4901.

Anthropic. (2025). Pricing. *Claude Platform Documentation*. Diakses pada 3 September 2026.

Anton, H., & Rorres, C. (2010). *Elementary Linear Algebra: Applications Version* (10th ed.). Hoboken: John Wiley & Sons.

Austin, J., Odena, A., Nye, M., Bosma, M., Michalewski, H., Dohan, D., Jiang, E., Cai, C., Terry, M., Le, Q., & Sutton, C. (2021). Program synthesis with large language models. *arXiv preprint* arXiv:2108.07732.

Bain, L. J., & Engelhardt, M. (1992). *Introduction to Probability and Mathematical Statistics* (2nd ed.). Boston: PWS-KENT.

Bishop, C. M. (2006). *Pattern Recognition and Machine Learning*. New York: Springer.

Boyd, S., & Vandenberghe, L. (2004). *Convex Optimization*. Cambridge: Cambridge University Press.

Casella, G., & Berger, R. L. (2002). *Statistical Inference* (2nd ed.). Pacific Grove: Duxbury.

Chen, M., Tworek, J., Jun, H., Yuan, Q., Pinto, H. P. de O., Kaplan, J., dkk. (2021). Evaluating large language models trained on code. *arXiv preprint* arXiv:2107.03374.

Chen, X., dkk. (2025). *Reasoning beyond language: A comprehensive survey on latent chain-of-thought reasoning*. arXiv:2505.16782.

Clark, P., Cowhey, I., Etzioni, O., Khot, T., Sabharwal, A., Schoenick, C., & Tafjord, O. (2018). Think you have solved question answering? Try ARC, the AI2 Reasoning Challenge. *arXiv preprint* arXiv:1803.05457.

Cobbe, K., Kosaraju, V., Bavarian, M., Chen, M., Jun, H., Kaiser, L., dkk. (2021). Training verifiers to solve math word problems. *arXiv preprint* arXiv:2110.14168.

Cover, T. M., & Thomas, J. A. (2006). *Elements of Information Theory* (2nd ed.). Hoboken: John Wiley & Sons.

Efron, B., & Tibshirani, R. J. (1993). *An Introduction to the Bootstrap*. New York: Chapman & Hall.

Fama, E. F., & French, K. R. (2015). A five-factor asset pricing model. *Journal of Financial Economics*, 116(1), 1–22.

Finin, T., Fritzson, R., McKay, D., & McEntire, R. (1994). KQML as an agent communication language. *Proceedings of the 3rd International Conference on Information and Knowledge Management (CIKM)*, 456–463.

Friedman, J. H. (2001). Greedy function approximation: A gradient boosting machine. *The Annals of Statistics*, 29(5), 1189–1232.

Fu, T., dkk. (2025). *Cache-to-Cache: Direct semantic communication between large language models*. arXiv:2510.03215.

Gelman, A., Carlin, J. B., Stern, H. S., Dunson, D. B., Vehtari, A., & Rubin, D. B. (2013). *Bayesian Data Analysis* (3rd ed.). Boca Raton: CRC Press.

Golub, G. H., & Van Loan, C. F. (2013). *Matrix Computations* (4th ed.). Baltimore: Johns Hopkins University Press.

Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. Cambridge: MIT Press.

Grinold, R. C., & Kahn, R. N. (2000). *Active Portfolio Management: A Quantitative Approach for Producing Superior Returns and Controlling Risk* (2nd ed.). New York: McGraw-Hill.

Gumbel, E. J. (1958). *Statistics of Extremes*. New York: Columbia University Press.

Guo, T., Chen, X., Wang, Y., Chang, R., Pei, S., Chawla, N. V., Wiest, O., & Zhang, X. (2024). *Large language model based multi-agents: A survey of progress and challenges*. arXiv:2402.01680.

Han, J., Zhang, S., Li, W., Dong, Y., Hu, T., dkk. (2026). *QuantaAlpha: An Evolutionary Framework for LLM-Driven Alpha Mining*. arXiv:2602.07085.

Hao, S., Sukhbaatar, S., Su, D., Li, X., Hu, Z., Weston, J., & Tian, Y. (2024). *Training large language models to reason in a continuous latent space*. arXiv:2412.06769.

Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The Elements of Statistical Learning: Data Mining, Inference, and Prediction* (2nd ed.). New York: Springer.

Hoerl, A. E., & Kennard, R. W. (1970). Ridge regression: Biased estimation for nonorthogonal problems. *Technometrics*, 12(1), 55–67.

Holtzman, A., Buys, J., Du, L., Forbes, M., & Choi, Y. (2020). The curious case of neural text degeneration. *International Conference on Learning Representations (ICLR)*.

IBM. (2025). 2025 Cost of a Data Breach Report: Navigating the AI rush without sidelining security. Armonk: IBM Corporation. Diakses pada 3 September 2026.

Jang, E., Gu, S., & Poole, B. (2017). Categorical reparameterization with Gumbel-Softmax. *International Conference on Learning Representations (ICLR)*.

Jensen, M. C. (1968). The performance of mutual funds in the period 1945–1964. *The Journal of Finance*, 23(2), 389–416.

Jin, D., Pan, E., Oufattole, N., Weng, W.-H., Fang, H., & Szolovits, P. (2021). What disease does this patient have? A large-scale open domain question answering dataset from medical exams. *Applied Sciences*, 11(14), 6421.

Johnson, R. A., & Wichern, D. W. (2007). *Applied Multivariate Statistical Analysis* (6th ed.). Upper Saddle River, NJ: Pearson Prentice Hall.

Kakushadze, Z. (2016). 101 formulaic alphas. *Wilmott*, 2016(84), 72–81.

Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q., & Liu, T.-Y. (2017). LightGBM: A highly efficient gradient boosting decision tree. *Advances in Neural Information Processing Systems*, 30, 3149–3157.

Kingma, D. P., & Welling, M. (2014). *Auto-encoding variational Bayes*. *Proceedings of the 2nd International Conference on Learning Representations (ICLR)*. arXiv:1312.6114.

Li, Y., Yang, X., Yang, X., Xu, M., Wang, X., Liu, W., & Bian, J. (2025). *R&D-Agent-Quant: A multi-agent framework for data-centric factors and model joint optimization*. arXiv:2505.15155.

Liu, J., Xia, C. S., Wang, Y., & Zhang, L. (2023). Is your code generated by ChatGPT really correct? Rigorous evaluation of large language models for code generation. *Advances in Neural Information Processing Systems (NeurIPS)*, 36.

Luce, R. D. (1959). *Individual Choice Behavior: A Theoretical Analysis*. New York: Wiley.

Maddison, C. J., Mnih, A., & Teh, Y. W. (2017). The Concrete distribution: A continuous relaxation of discrete random variables. *International Conference on Learning Representations (ICLR)*.

Magnus, J. R., & Neudecker, H. (1988). *Matrix Differential Calculus with Applications in Statistics and Econometrics*. Chichester: John Wiley & Sons.

Markowitz, H. (1952). Portfolio selection. *The Journal of Finance*, 7(1), 77–91.

McNemar, Q. (1947). Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika*, 12(2), 153–157.

Zhuang, Y., Liu, S., Cheng, S., Zhang, Y., Wang, X., & Zhang, C. (2025). *Mixture of inputs: Text generation beyond discrete token sampling*. arXiv:2505.14827. Dipresentasikan pada *Advances in Neural Information Processing Systems* (NeurIPS 2025).

Mikolov, T., Chen, K., Corrado, G., & Dean, J. (2013). *Efficient estimation of word representations in vector space*. arXiv:1301.3781.

nostalgebraist. (2020). Interpreting GPT: The logit lens. *LessWrong*.

OpenAI. (2026). Pricing. *OpenAI API Documentation*. Diakses pada 3 September 2026.

Park, K., Choe, Y. J., & Veitch, V. (2024). *The linear representation hypothesis and the geometry of large language models*. Proceedings of the 41st International Conference on Machine Learning (ICML). arXiv:2311.03658.

Rein, D., Hou, B. L., Stickland, A. C., Petty, J., Pang, R. Y., Dirani, J., Michael, J., & Bowman, S. R. (2024). GPQA: A graduate-level Google-proof Q&A benchmark. *Conference on Language Modeling (COLM)*.

Russell, S., & Norvig, P. (2021). *Artificial Intelligence: A Modern Approach* (4th ed.). Hoboken, NJ: Pearson.

Searle, S. R., & Khuri, A. I. (2017). *Matrix Algebra Useful for Statistics* (2nd ed.). Hoboken: John Wiley & Sons.

Sennrich, R., Haddow, B., & Birch, A. (2016). Neural machine translation of rare words with subword units. *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (ACL)*, 1715–1725.

Shannon, C. E. (1948). A mathematical theory of communication. *The Bell System Technical Journal*, 27(3), 379–423.

Sharpe, W. F. (1964). Capital asset prices: A theory of market equilibrium under conditions of risk. *The Journal of Finance*, 19(3), 425–442.

Sharpe, W. F. (1994). The Sharpe ratio. *The Journal of Portfolio Management*, 21(1), 49–58.

Spearman, C. (1904a). The proof and measurement of association between two things. *The American Journal of Psychology*, 15(1), 72–101.

Spearman, C. (1904b). "General intelligence," objectively determined and measured. *The American Journal of Psychology*, 15(2), 201–292.

Stanford Institute for Human-Centered Artificial Intelligence. (2025). *The 2025 AI Index Report*. Stanford: Stanford University. Diakses pada 3 September 2026.

Stanford Institute for Human-Centered Artificial Intelligence. (2026). *The 2026 AI Index Report: Research and development*. Stanford: Stanford University. Diakses pada 3 September 2026.

Su, J., Ahmed, M., Lu, Y., Pan, S., Bo, W., & Liu, Y. (2024). RoFormer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 568, 127063.

Subanar. (2013). *Statistika Matematika*. Yogyakarta: Graha Ilmu.

Tandelilin, E. (2010). *Portofolio dan Investasi: Teori dan Aplikasi* (Edisi Pertama). Yogyakarta: Kanisius.

Tang, Z., Chen, Z., Yang, J., Mai, J., Zheng, Y., Wang, K., Chen, J., & Lin, L. (2025). *AlphaAgent: LLM-driven alpha mining with regularized exploration to counteract alpha decay*. arXiv:2502.16789.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., & Polosukhin, I. (2017). *Attention is all you need*. *Advances in Neural Information Processing Systems*, 30.

Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., Chi, E. H., Le, Q. V., & Zhou, D. (2022). Chain-of-thought prompting elicits reasoning in large language models. *Advances in Neural Information Processing Systems*, 35, 24824–24837.

Wooldridge, M., & Jennings, N. R. (1995). Intelligent agents: Theory and practice. *The Knowledge Engineering Review*, 10(2), 115–152.

Wooldridge, M. (2009). *An Introduction to MultiAgent Systems* (2nd ed.). Chichester: Wiley.

Wu, C., dkk. (2025). LLMs are single-threaded reasoners: Demystifying the working mechanism of soft thinking. *arXiv preprint* arXiv:2508.03440.

Yang, X., Liu, W., Zhou, D., Bian, J., & Liu, T.-Y. (2020). *Qlib: An AI-oriented quantitative investment platform*. arXiv:2009.11189.

Yang, A., dkk. (Qwen Team). (2025). *Qwen3 technical report*. arXiv:2505.09388.

Yu, S., Xue, H., Ao, X., Pan, F., He, J., Tu, D., & He, Q. (2023). Generating synergistic formulaic alpha collections via reinforcement learning. *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining* (KDD '23).

Zhang, Z., He, X., Yan, W., Shen, A., Zhao, C., Wang, S., Shen, Y., & Wang, X. E. (2025). *Soft thinking: Unlocking the reasoning potential of LLMs in continuous concept space*. arXiv:2505.15778.

Zou, J., Qiu, R., Li, G., Yang, X., Tieu, K., Lu, P., Shen, K., Tong, H., Choi, Y., He, J., Zou, J., Wang, M., & Yang, L. (2025). *Latent collaboration in multi-agent systems*. arXiv:2511.20639.

PyTorch Contributors. (2026). *torch.cat*. *PyTorch Documentation*. Diakses pada 22 Oktober 2026.